Practical AI and Machine Learning for Materials Science



Session 6: LLMs, agents and the end-to-end workflow · Taylor D. Sparks

Last session: four things to remember

1. Fewer experiments, not perfect predictions

The model only has to rank where to look

2. Explore on purpose

Greedy search gets stuck on the first peak

3. Match the loop to your lab

Batches, constraints, several goals

4. Trade-offs are decisions

The Pareto front shows the options; you choose

Today: language models as the glue around models you can trust

Today answers six of your questions

Mining the literature
Extraction you can measure; KnowMat

Local or cloud LLM?
Data, cost, quality: how to decide

Foundation models
A universal potential, tested on our data

Knowledge graphs
Where they fit with LLMs

How was this course built?
An AI coding agent, checked like a thesis

Choosing methods
Confidence for newcomers and for industry

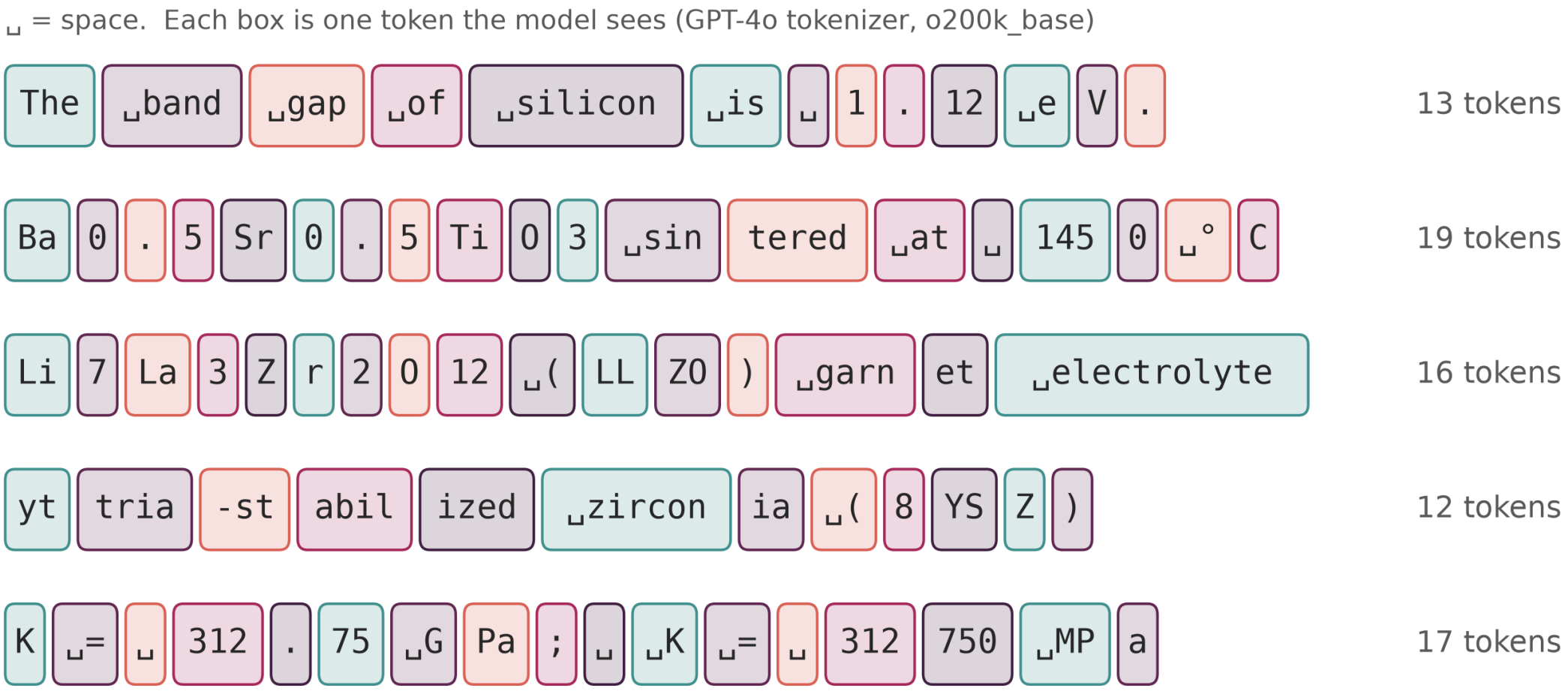
Key idea: LLMs are the glue, not the source of the numbers

How LLMs work

Tokens, probabilities, training, embeddings, and how we fix their failures

LLMs read tokens, not letters: 1450 °C arrives as “145”, “0”, “ °”, “C”

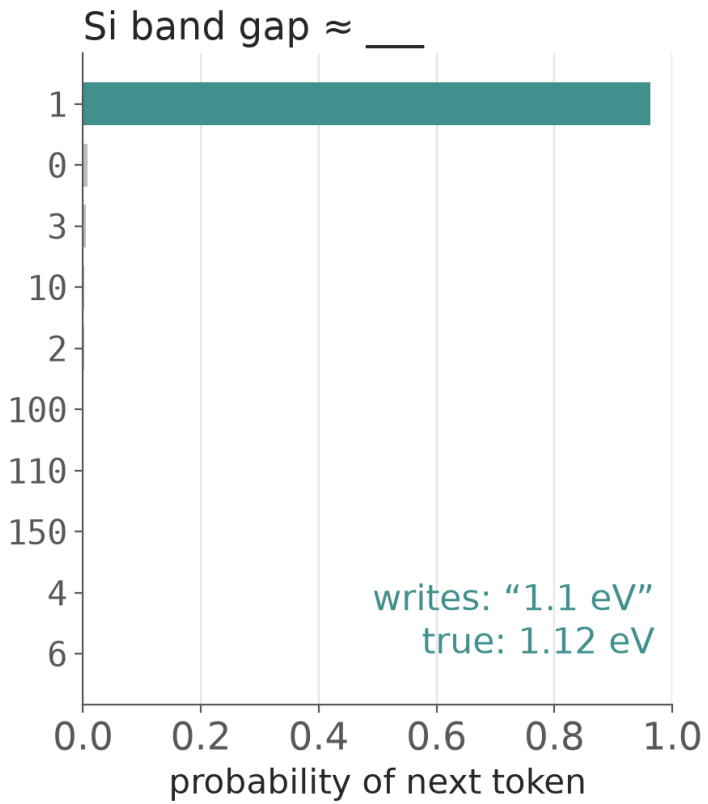
Formulas and numbers are split arbitrarily, one reason LLMs slip on digits, units and stoichiometry

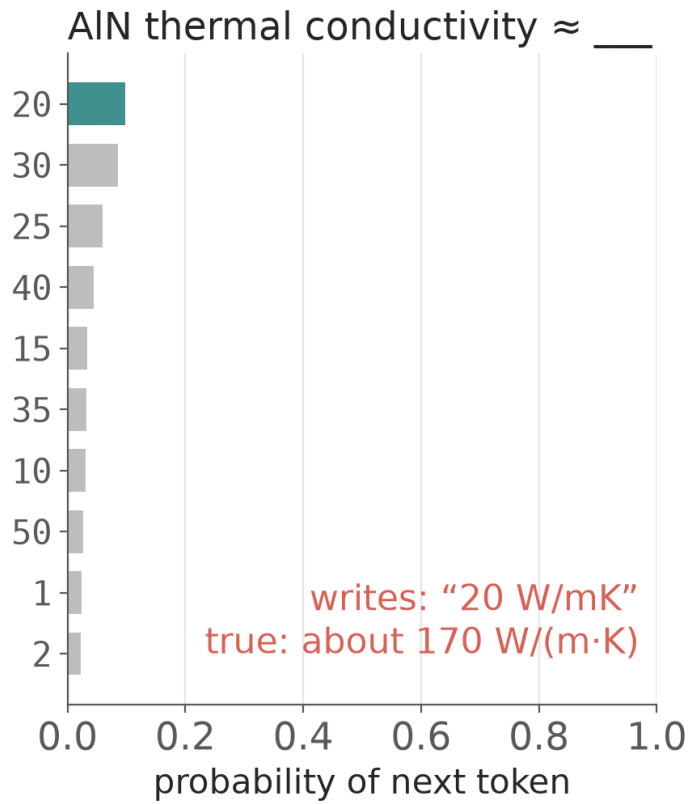


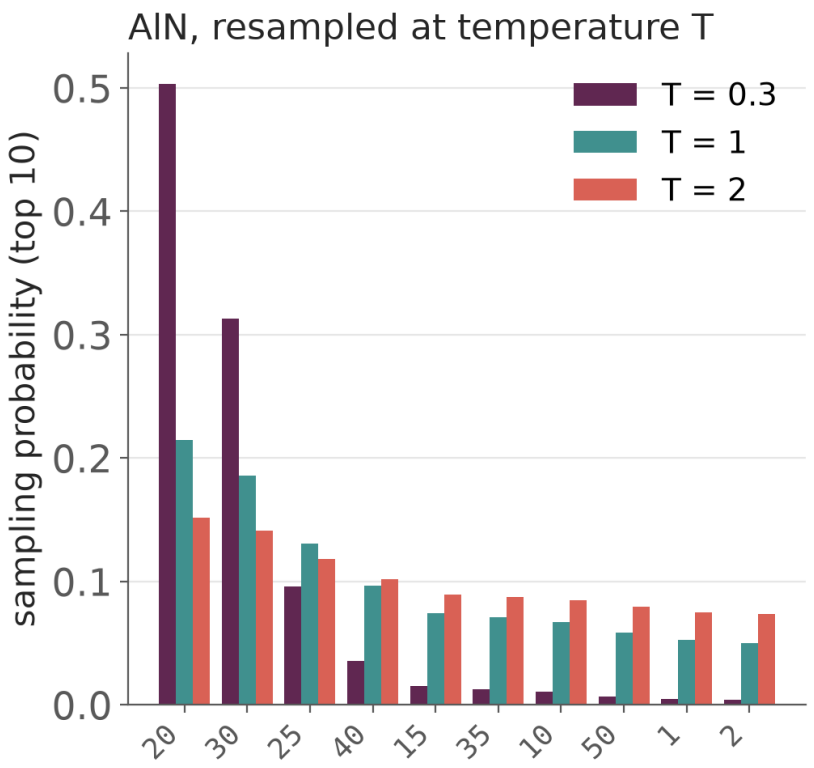
Tokenizer: OpenAI o200k_base (GPT-4o) via tiktoken. Every model family has its own tokenizer. Made for this course.

An LLM's answer is a probability distribution; fluent text hides it

Silicon: 96% on the right first digit. AlN: the top guess gets 10%, and the model writes "20 W/mK" anyway







llama3.2:3b running locally in Ollama, next-token log-probabilities (logprobs=True). Made for this course.

A chat model is a next-token predictor, then taught to help and reason

1. Pretrain

Predict the next token on 10-30 trillion tokens of web, books, code, papers

Knowledge and language. Months on thousands of GPUs

→

2. Instruction-tune

Fine-tune on curated prompt → answer pairs

Follows instructions, formats, chat

→

3. Preferences + RL

Humans rank answers (RLHF); math and code graded automatically (RLVR)

Helpful, harmless, and right on checkable tasks

→

4. Reason at run time

Writes intermediate steps before the answer

More thinking = more tokens = better on hard problems, slower, costlier

Where you come in: a fine-tune (our CrysText, Dagdelen et al.) is a small step 2 on someone else's step 1. The notebook's `effort` setting is the step-4 knob.

Token counts: e.g. Llama 3 (15T tokens, Meta 2024), Qwen3 (36T, 2025). RLVR: reinforcement learning from verifiable rewards.

On chemistry exams the best LLMs beat expert chemists, on average

ChemBench: 2,788 questions, from textbook to research-level, vs 19 chemists

Better on average

The best models outscored the best human chemist in the study

Overconfident

Most models are poorly calibrated: their stated confidence misleads

Still trip on basics

Struggle with some simple tasks, e.g. reasoning about structures

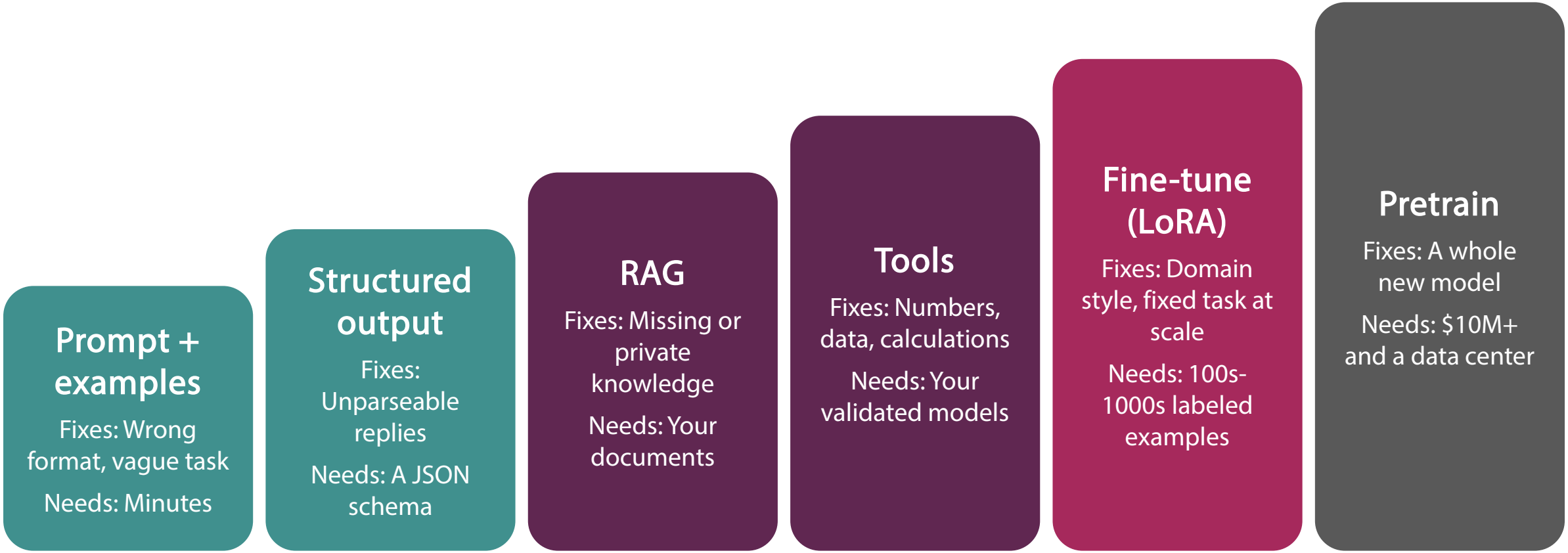
Strong enough to use; not calibrated enough to trust without your own answer key

Mirza et al., Nat. Chem. 17, 1027 (2025) https://doi.org/10.1038/s41557-025-01815-x

LLMs are powerful, but they fail in predictable ways

- Hallucinations – confidence but incorrect outputs
- Numerical errors – struggling with exact values
- Overconfidence – not expressing uncertainty well
- Inconsistencies – same prompt, different answers

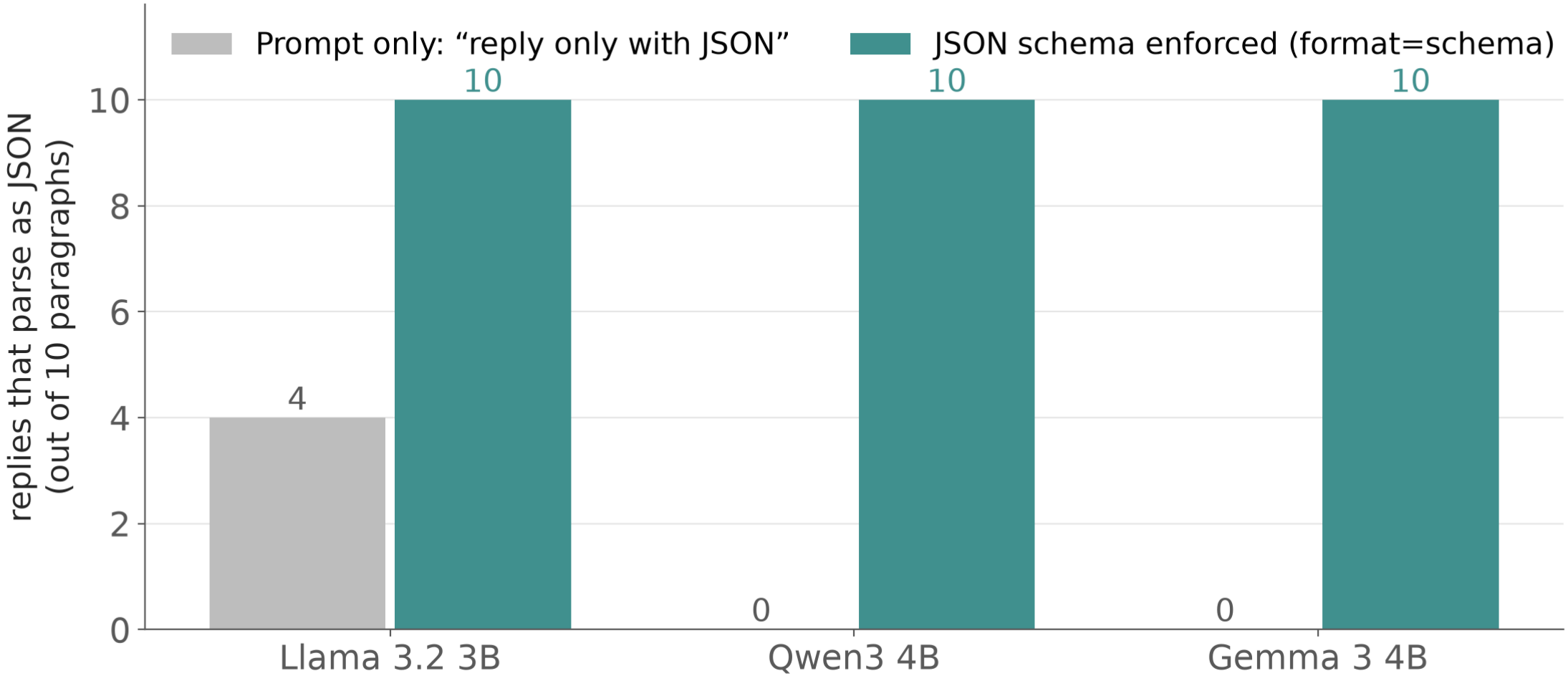
Fix a model's gaps with the cheapest step that works



Most materials projects stop at tools; fine-tune only for one fixed task run thousands of times

A JSON schema made every local reply parse; asking nicely: 0-4 of 10

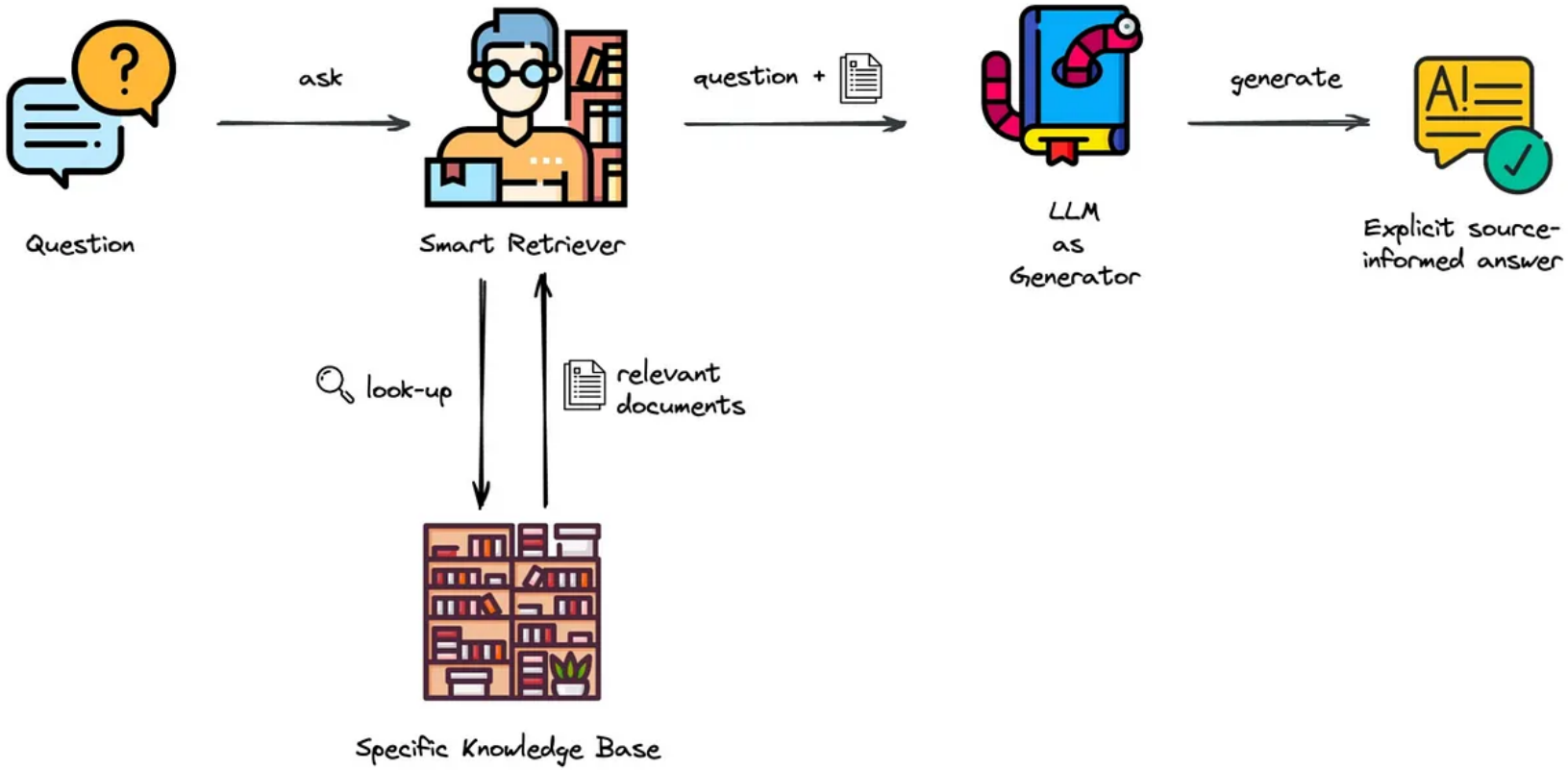
Constrained decoding masks tokens that would break the schema: shape guaranteed, values not



Same 10 paragraphs, prompt and schema as notebook Part A; local models via Ollama, temperature 0. Made for this course.

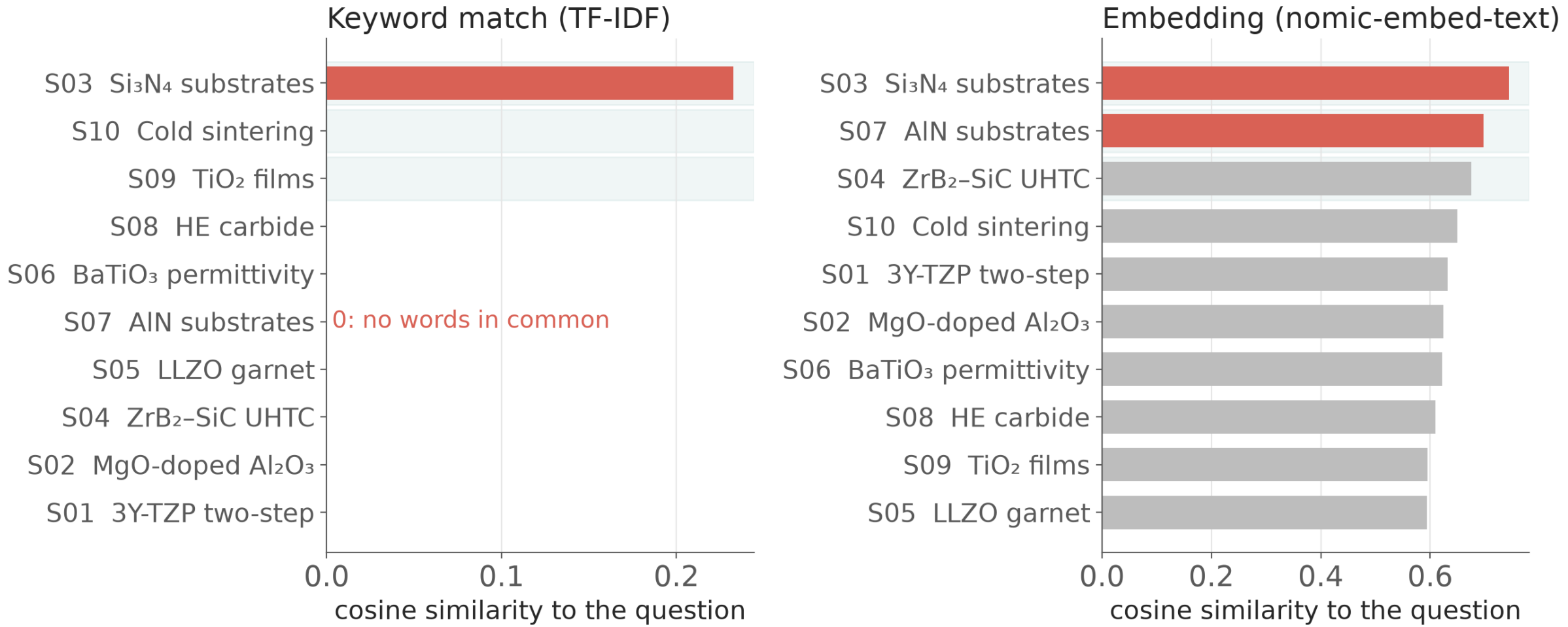
Retrieval Augmented Generation (RAG) offers solutions to LLM problems

LLMs have token size limitations and training data cutoff date.



Embeddings find the AlN passage that keyword search scored zero

Notebook Part B's question. Keyword score for AlN: 0. Embedding: AlN ranks 2, from a 274 MB local model



Shaded: the top 3 passages handed to the LLM. Coral: the two correct substrate passages (AlN, Si3N4)

nomic-embed-text v1.5 via Ollama vs the notebook's TF-IDF retriever. Made for this course.

Tool calling is another powerful way to fix LLM shortcomings

"Convert 30,000 BTU's to Joules"

```
{
  "name": "convert_units",
  "description": "Convert a value between units",
  "parameters": {
    "value": "number",
    "from_unit": "string",
    "to_unit": "string"
  }
}
```

About Pint

Units in Python. You are currently looking at the documentation of version 0.6.

Other Formats

You can download the documentation in other formats as well:

as PDF
as zipped HTML
as ePub

Useful Links

Pint @ PyPI
Code in GitHub
Issue Tracker

Pint: a Python units library

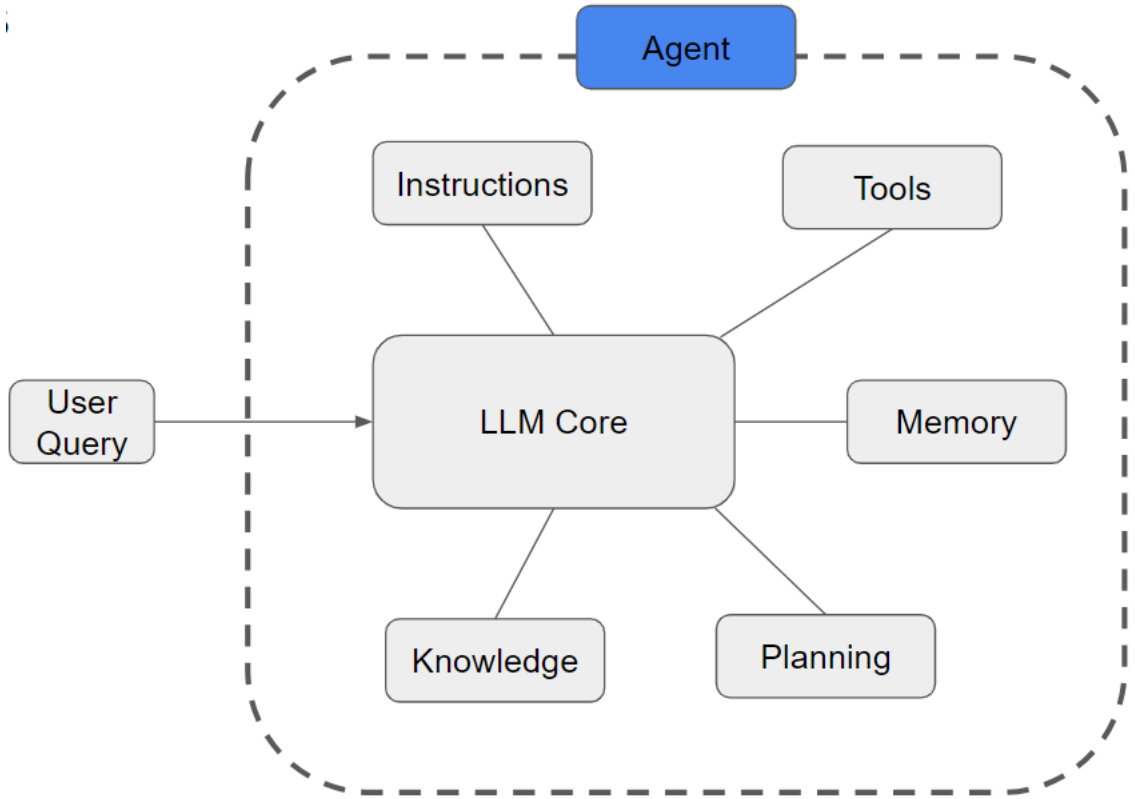
Pint is Python package to define, operate and manipulate **physical quantities**: the product of a numerical value and a unit of measurement. It allows arithmetic operations between them and conversions from and to different units.

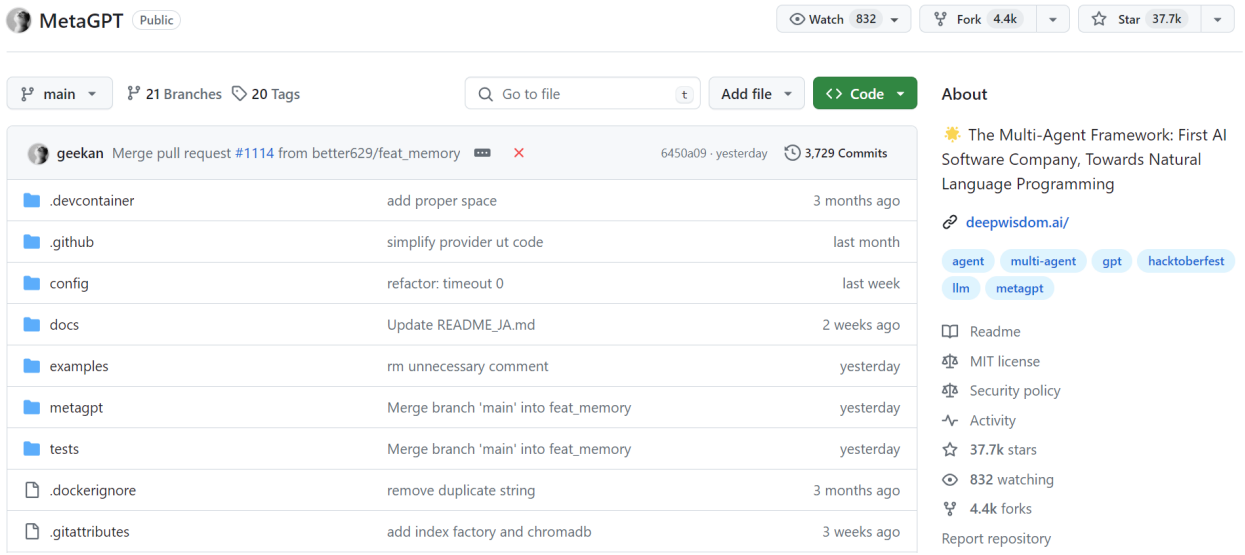


It is distributed with a comprehensive list of physical units, prefixes and constants. Due to its modular design, you can extend (or even rewrite!) the complete list without changing the source code. It supports a lot of numpy mathematical operations without monkey patching or wrapping numpy.

It has a complete test coverage. It runs in Python 2.6+ and 3.2+ with no other dependency. It licensed under BSD.

Agents offer another application of LLMs





Mining the literature

From prose to a database you can trust

Now let's discuss how LLMs are being used in materials science

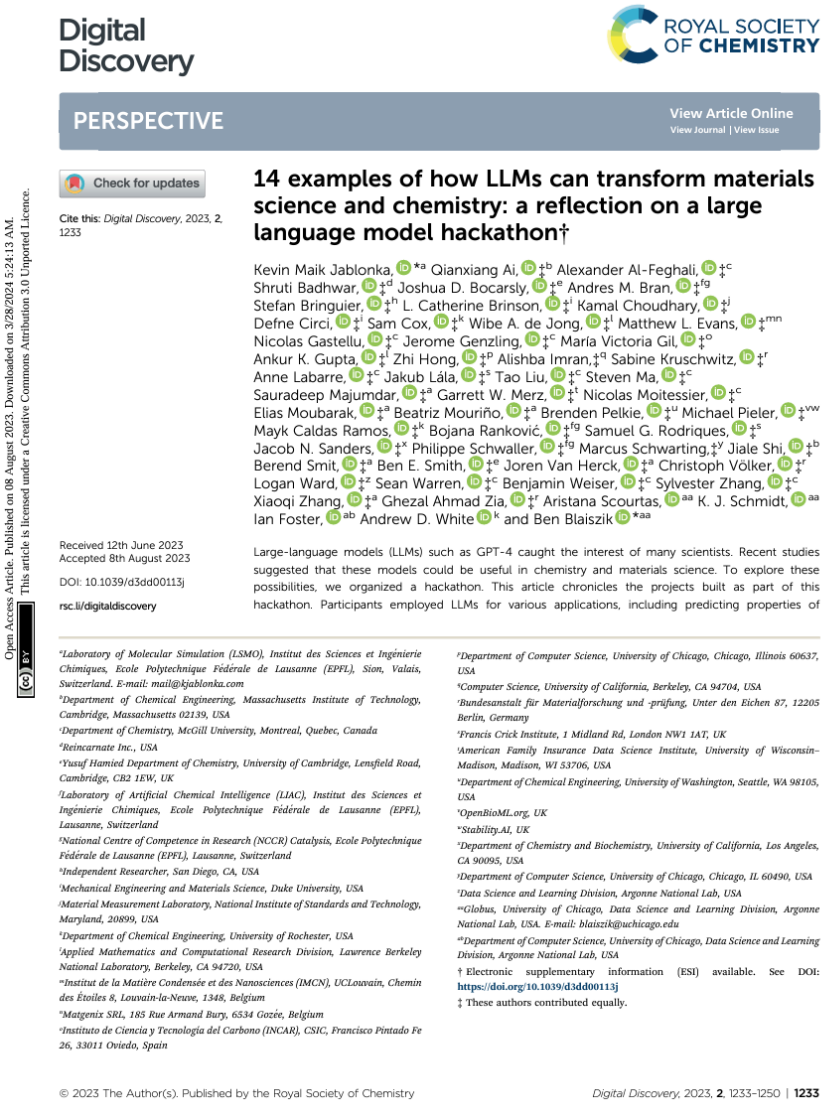
Data extraction

Property prediction

Generative models

Automation and novel interfaces

Education



Digital Discovery

ROYAL SOCIETY OF CHEMISTRY

PERSPECTIVE

View Article Online
View Journal | View Issue

Check for updates

Cite this: *Digital Discovery*, 2023, 2, 1233

14 examples of how LLMs can transform materials science and chemistry: a reflection on a large language model hackathon†

Kevin Maik Jablonka, *[a] Qianxiang Ai, [b] Alexander Al-Feghali, [c] Shruti Badhwar, [d] Joshua D. Bocarsly, [e] Andres M. Bran, [fg] Stefan Bringuier, [h] L. Catherine Brinson, [i] Kamal Choudhary, [j] Defne Circi, [l] Sam Cox, [k] Wibe A. de Jong, [l] Matthew L. Evans, [mn] Nicolas Gastellu, [c] Jerome Genzling, [c] María Victoria Gil, [o] Ankur K. Gupta, [l] Zhi Hong, [p] Alishba Imran, [q] Sabine Kruschwitz, [r] Anne Labarre, [c] Jakub Lála, [s] Tao Liu, [c] Steven Ma, [c] Sauradeep Majumdar, [a] Garrett W. Merz, [t] Nicolas Moitessier, [c] Elias Moubarak, [a] Beatriz Mouriño, [a] Brenden Pelkie, [u] Michael Pieler, [vw] Mayk Caldas Ramos, [k] Bojana Ranković, [fg] Samuel G. Rodriques, [s] Jacob N. Sanders, [x] Philippe Schwaller, [fg] Marcus Schwarting, [y] Jiale Shi, [b] Berend Smit, [a] Ben E. Smith, [e] Joren Van Herck, [a] Christoph Völker, [r] Logan Ward, [z] Sean Warren, [c] Benjamin Weiser, [c] Sylvester Zhang, [c] Xiaoqi Zhang, [a] Ghezal Ahmad Zia, [r] Aristana Scourtas, [aa] K. J. Schmidt, [aa] Ian Foster, [ab] Andrew D. White [k] and Ben Blaiszik *[aa]

Received 12th June 2023
Accepted 8th August 2023

DOI: 10.1039/d3dd00113j

rsc.li/digitaldiscovery

Large-language models (LLMs) such as GPT-4 caught the interest of many scientists. Recent studies suggested that these models could be useful in chemistry and materials science. To explore these possibilities, we organized a hackathon. This article chronicles the projects built as part of this hackathon. Participants employed LLMs for various applications, including predicting properties of

Open Access Article. Published on 08 August 2023. Downloaded on 3/28/2024 5:24:13 AM.
This article is licensed under a Creative Commons Attribution 3.0 Unported Licence.

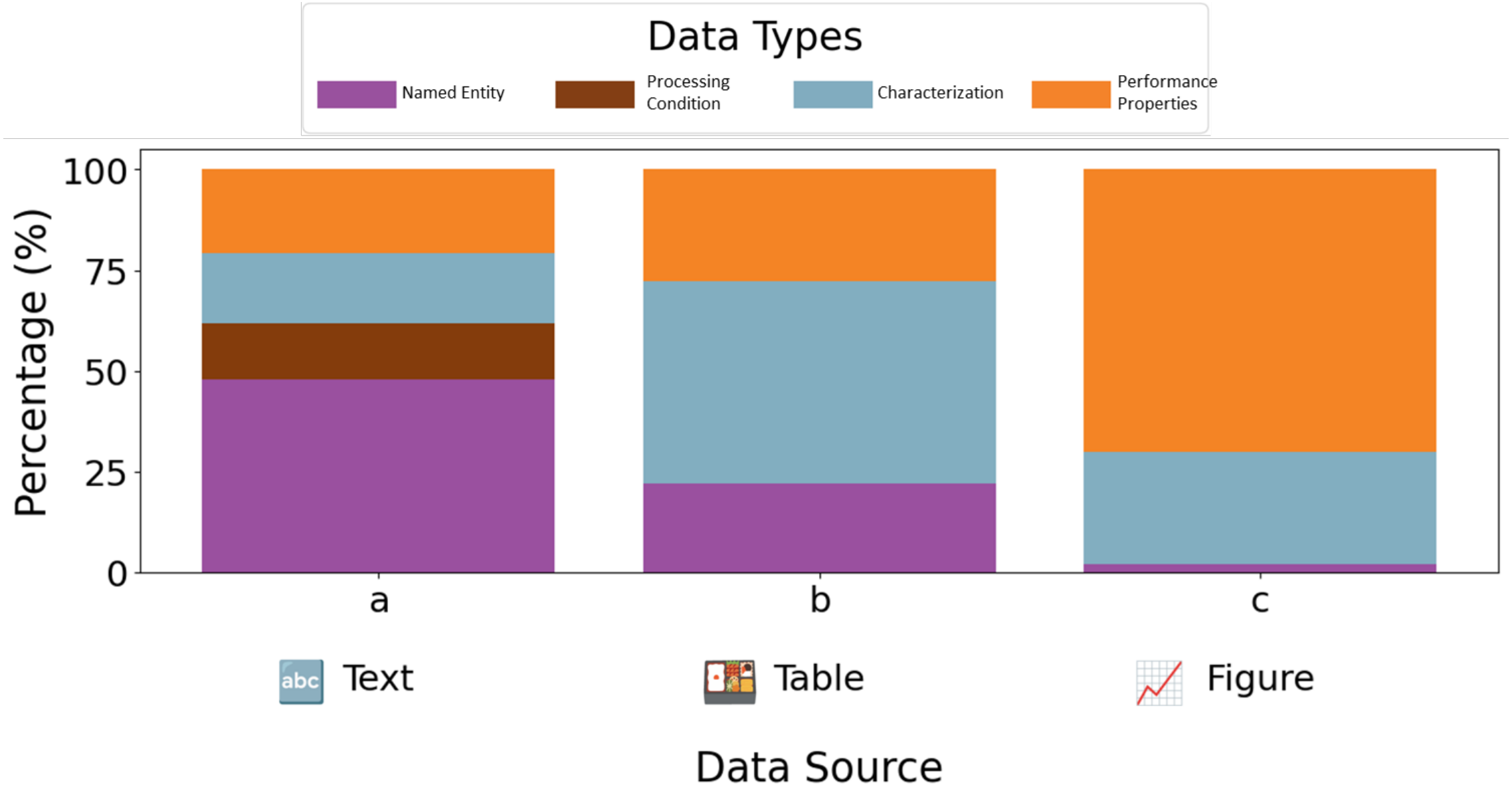
[a]Laboratory of Molecular Simulation (LSMO), Institut des Sciences et Ingénierie Chimiques, Ecole Polytechnique Fédérale de Lausanne (EPFL), Sion, Valais, Switzerland. E-mail: mail@kjablonka.com
[b]Department of Chemical Engineering, Massachusetts Institute of Technology, Cambridge, Massachusetts 02139, USA
[c]Department of Chemistry, McGill University, Montreal, Quebec, Canada
[d]Reincarnate Inc., USA
[e]Yusuf Hamied Department of Chemistry, University of Cambridge, Lensfield Road, Cambridge, CB2 1EW, UK
[f]Laboratory of Artificial Chemical Intelligence (LIAC), Institut des Sciences et Ingénierie Chimiques, Ecole Polytechnique Fédérale de Lausanne (EPFL), Lausanne, Switzerland
[g]National Centre of Competence in Research (NCCR) Catalysis, Ecole Polytechnique Fédérale de Lausanne (EPFL), Lausanne, Switzerland
[h]Independent Researcher, San Diego, CA, USA
[i]Mechanical Engineering and Materials Science, Duke University, USA
[j]Material Measurement Laboratory, National Institute of Standards and Technology, Maryland, 20899, USA
[k]Department of Chemical Engineering, University of Rochester, USA
[l]Applied Mathematics and Computational Research Division, Lawrence Berkeley National Laboratory, Berkeley, CA 94720, USA
[m]Institut de la Matière Condensée et des Nanosciences (IMCN), UCLouvain, Chemin des Étoiles 8, Louvain-la-Neuve, 1348, Belgium
[n]Matgenix SRL, 185 Rue Armand Bury, 6534 Gozée, Belgium
[o]Instituto de Ciencia y Tecnología del Carbono (INCAR), CSIC, Francisco Pintado Fe 26, 33011 Oviedo, Spain
[p]Department of Computer Science, University of Chicago, Chicago, Illinois 60637, USA
[q]Computer Science, University of California, Berkeley, CA 94704, USA
[r]Bundesanstalt für Materialforschung und -prüfung, Unter den Eichen 87, 12205 Berlin, Germany
[s]Francis Crick Institute, 1 Midland Rd, London NW1 1AT, UK
[t]American Family Insurance Data Science Institute, University of Wisconsin–Madison, Madison, WI 53706, USA
[u]Department of Chemical Engineering, University of Washington, Seattle, WA 98105, USA
[v]OpenBioML.org, UK
[w]Stability.AI, UK
[x]Department of Chemistry and Biochemistry, University of California, Los Angeles, CA 90095, USA
[y]Department of Computer Science, University of Chicago, Chicago, IL 60490, USA
[z]Data Science and Learning Division, Argonne National Lab, USA
[aa]Globus, University of Chicago, Data Science and Learning Division, Argonne National Lab, USA. E-mail: blaiszik@uchicago.edu
[ab]Department of Computer Science, University of Chicago, Data Science and Learning Division, Argonne National Lab, USA

† Electronic supplementary information (ESI) available. See DOI: https://doi.org/10.1039/d3dd00113j

‡ These authors contributed equally.

© 2023 The Author(s). Published by the Royal Society of Chemistry

Different data is stored in text / table / figure. NLP ignores figures



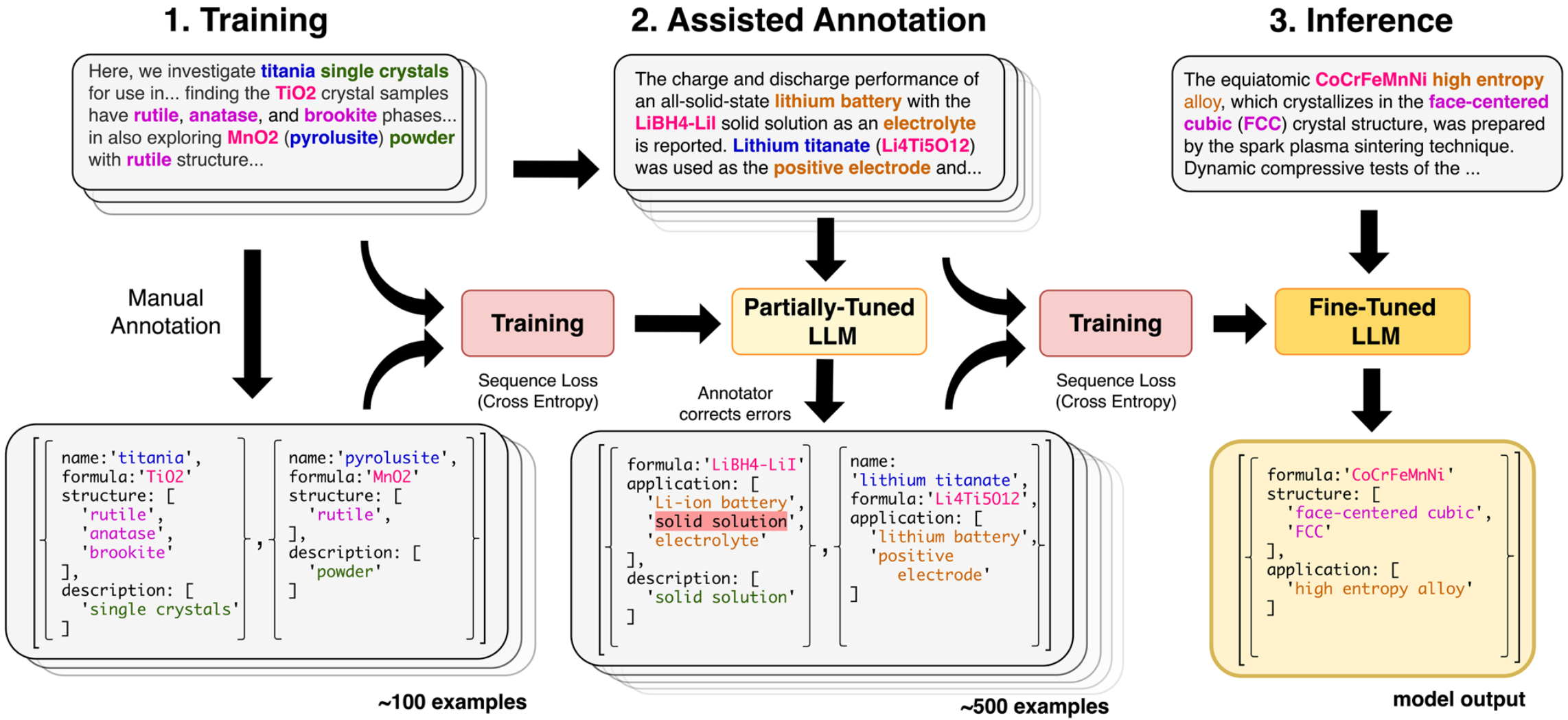
Score extraction against an answer key, or you don't know it works

Regex precision	Regex recall	LLM + JSON schema
0.74	0.88	measure it
28 of 38 values right	of 32 true records	same answer key, in the notebook

Keep an evidence quote for every value; spot-check a random sample

Fine-tuned LLMs can turn abstracts into structured records

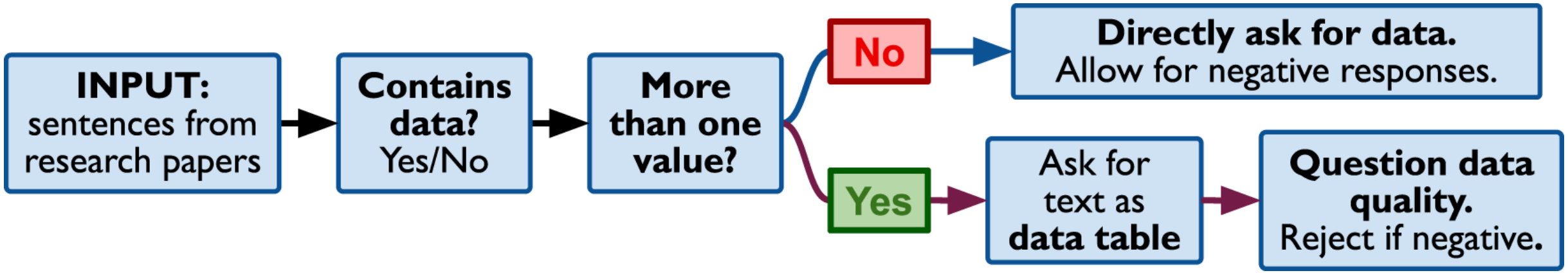
Model-assisted annotation cut expert time per abstract by 57%; best dopant-task F1 was 0.821



Dagdelen et al., Nat. Commun. 15, 1418 (2024) https://doi.org/10.1038/s41467-024-45563-x

Follow-up questions keep LLM data extraction honest

GPT-4 with ChatExtract: 90.8% precision and 87.7% recall on bulk modulus, with no fine-tuning



Polak & Morgan, Nat. Commun. 15, 1569 (2024) https://doi.org/10.1038/s41467-024-45914-8

Vanilla LLMs…. Leave much to be desired!

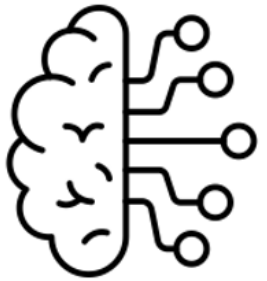
Prompt

Extract material properties as JSON:

```
{  "material": "...",
   "properties": [
       {"name": "...",
        "value": "...",
        "unit": "..."}
   ]
}
```

Input:



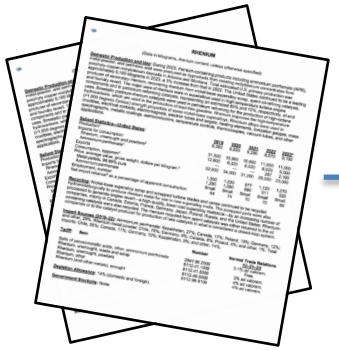


Vanilla LLM



- Hallucinations
- No validation or correction
- Inconsistent schema compliance

Our solution: Iterative multi-agent workflow



Paper
PDF

Parser Agent

Extraction Agent

Evaluator Agent

Re-run if needed (≤3×)

With updated prompt

Manager Agent

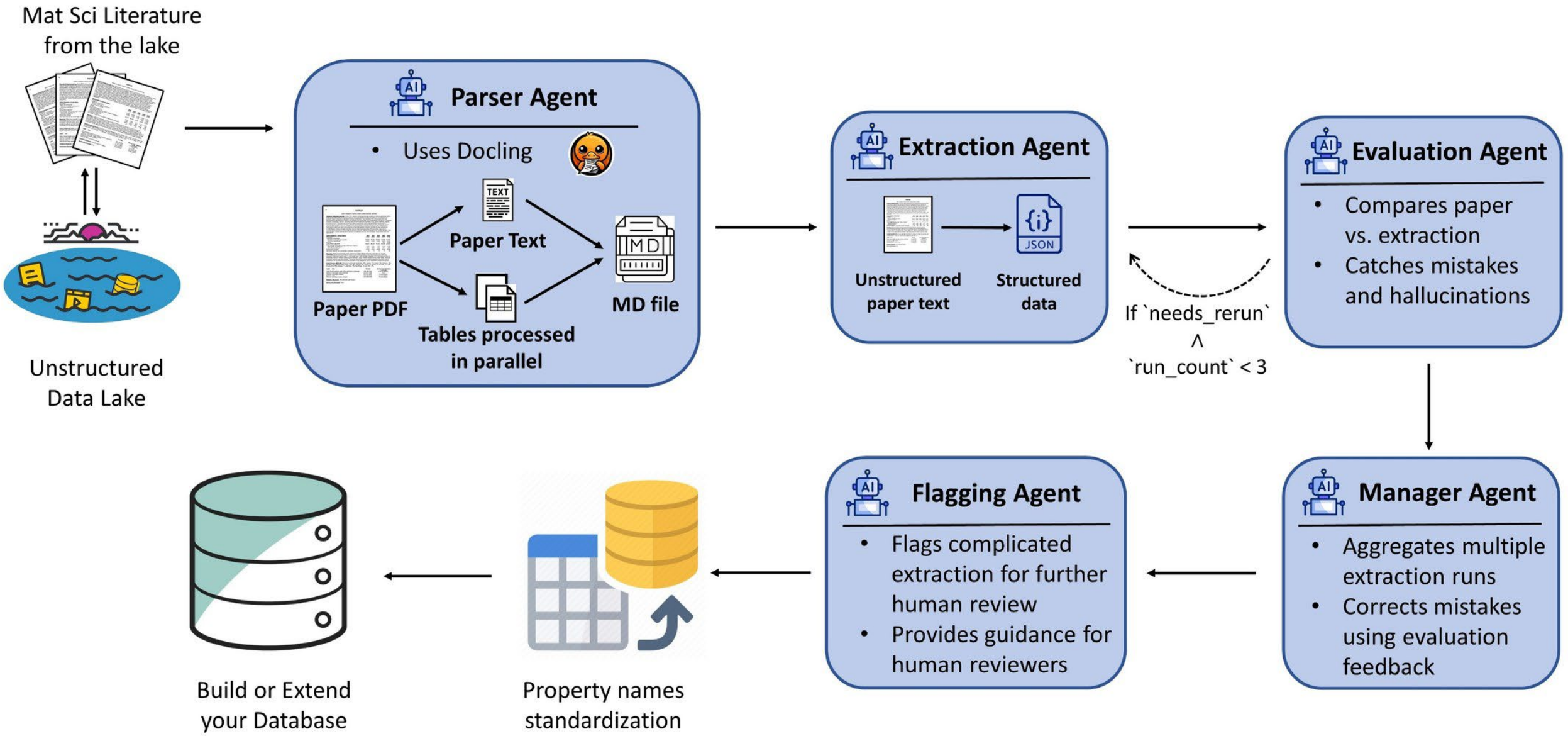
Flagging Agent

{i}
JSON

Final
JSON

Database

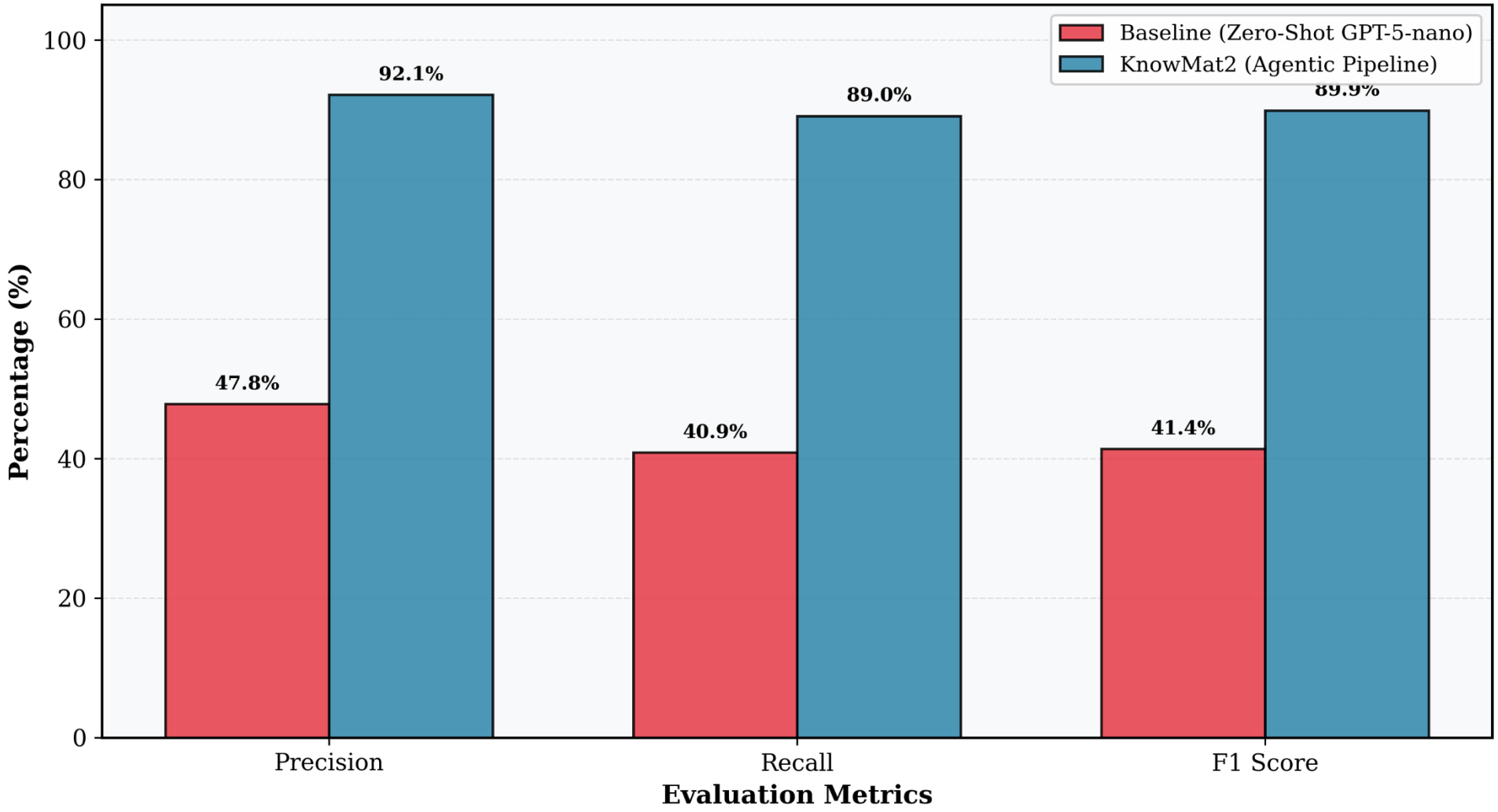
Complete system is a multi-agent framework



Multi-agent is far superior to Vanilla LLM and structured prompts

Strict Matching:

- ✓ Composition
- ✓ Property Name
- ✓ Value
- ✓ Unit



KnowMat is ready for materials data extraction!

KnowMat: An Agentic Approach to Transforming Unstructured Material Science Literature into Structured Data

Hasan M. Sayeed[1], Casey Clark[1], Trupti Mohanty[1], and Taylor D. Sparks *[1]

[1]Department of Materials Science & Engineering, University of Utah, Salt Lake City, UT 84112, USA

Abstract

The materials-science literature is the richest reservoir of domain knowledge, yet converting its unstructured text—especially narrative passages and complex tables—into machine-readable data for analysis and ML model training remains challenging. To address this, we present KnowMat, an agentic, multi-stage pipeline that transforms full-text articles into schema-aligned, machine-readable JSON. KnowMat parses PDFs (text and tables), and performs iterative extraction with evaluation-driven re-runs to enhance coverage while curbing hallucinations. A two-stage manager then aggregates, validates, and corrects results, while properties are encoded with a fidelity-preserving dual representation (original textual form along with numeric surrogate with explicit value type); standardized labels are added without altering author-reported names to support database integration. Although demonstrated for materials literature, the workflow is schema-agnostic and readily adaptable to other scientific domains. Evaluation on real-world materials science papers demonstrates KnowMat's accuracy and efficiency, significantly reducing barriers to data-driven materials research.

1 Introduction

The field of machine learning (ML) for materials science faces a significant challenge due to the scarcity of high-quality data. Despite the rapid growth in research output, much of the experimental data remains locked within the confines of literature, which is often non-machine-readable and thus inaccessible for large-scale computational analysis [1, 2]. This limitation stifles the development of advanced ML models, as they rely heavily on rich, diverse datasets to achieve meaningful results.

Existing open-source databases in materials science predominantly focus on computationally derived properties [3–6]. While valuable, these datasets introduce potential systematic errors and are constrained to quantities that can be rapidly computed, leaving a gap in real-world, experimental data representation. Datasets extracted manually from literature do exist, but they are often proprietary, requiring significant financial and labor investments [7–9]. This exclusivity further hampers accessibility and widespread use.

Efforts to address these challenges have led to the development of various data extraction approaches. Rule-based methods like ChemDataExtractor, OSCAR, LeadMine, ChemicalTagger offer structured data extraction using predefined rules and ontologies [10–13], but they are limited in scalability and adaptability. Advancements in language models (LMs) have enabled data extraction tailored to field-specific tasks. Domain-specific pre-training and fine-tuning of LMs on manually annotated corpora have proven effective in domain-specific tasks [14–18]. With advances in large language models (LLMs), this has dramatically changed because LLMs can solve tasks for which they have not been explicitly trained. LLMs thus present a powerful and scalable alternative for structured data extraction. LLMs like GPT-4 and LLaMA-2, combined with prompt engineering techniques, have demonstrated remarkable success in extracting structured data directly from literature [19–28]. These methods not only identify relevant information but also ensure high precision and recall in the resulting datasets [24, 27].

For wider adoption, there is a growing need for general-purpose pipelines capable of transforming extracted data into structured formats suitable for database integration. Such databases can then serve as foundational resources for downstream ML models, fostering advances in materials discovery and property prediction. This growing interest in literature-to-data pipelines underscores a broader paradigm shift toward automating the capture and structuring of scientific knowledge.

Building upon this vision, KnowMat is developed as a schema-driven, multi-agent system designed to automate the transformation of unstructured materials science literature into machine-readable datasets. KnowMat employs an agentic architecture powered by GPT-based models orchestrated through the LangGraph framework. Specialized agents handle each stage of the workflow—from document parsing to data extraction, validation, and aggregation—ensuring modularity and robustness. By enforcing schema-constrained extraction using TrustCall-style tool invocation, KnowMat guarantees that outputs conform to predefined Pydantic models, triggering automatic corrections when struc-

*Corresponding author: sparks@eng.utah.edu

https://doi.org/10.26434/chemrxiv-2025-l296q-v2 ORCID: https://orcid.org/0000-0002-6583-7755 Content not peer-reviewed by ChemRxiv. License: CC BY 4.0





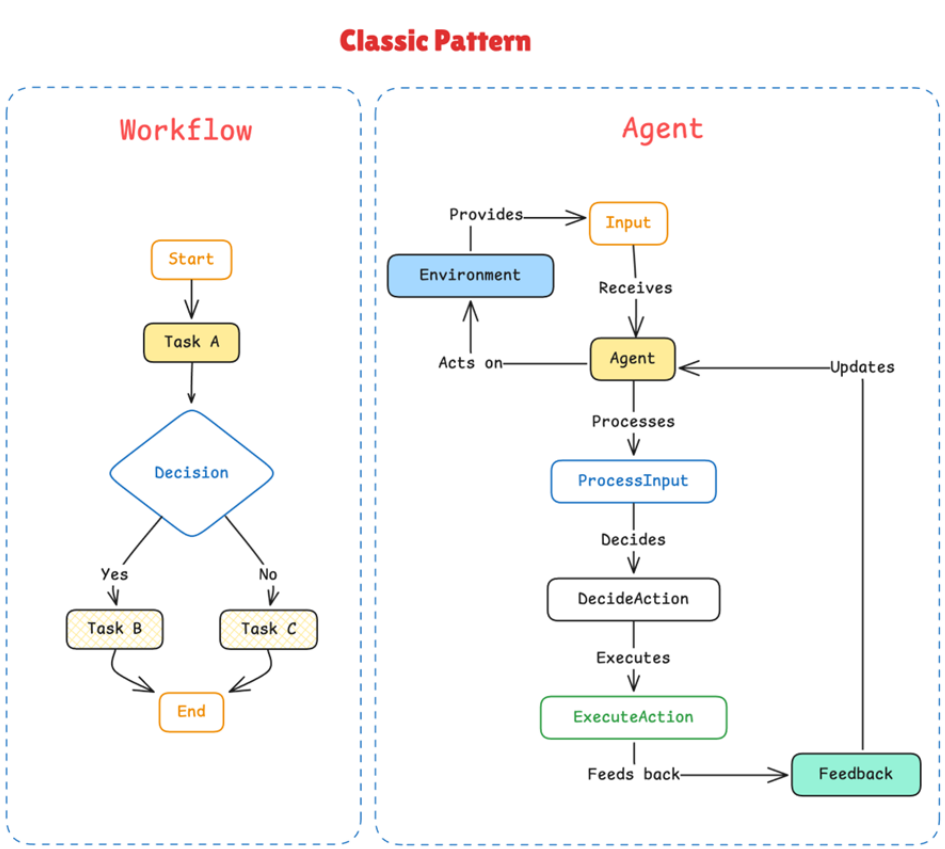


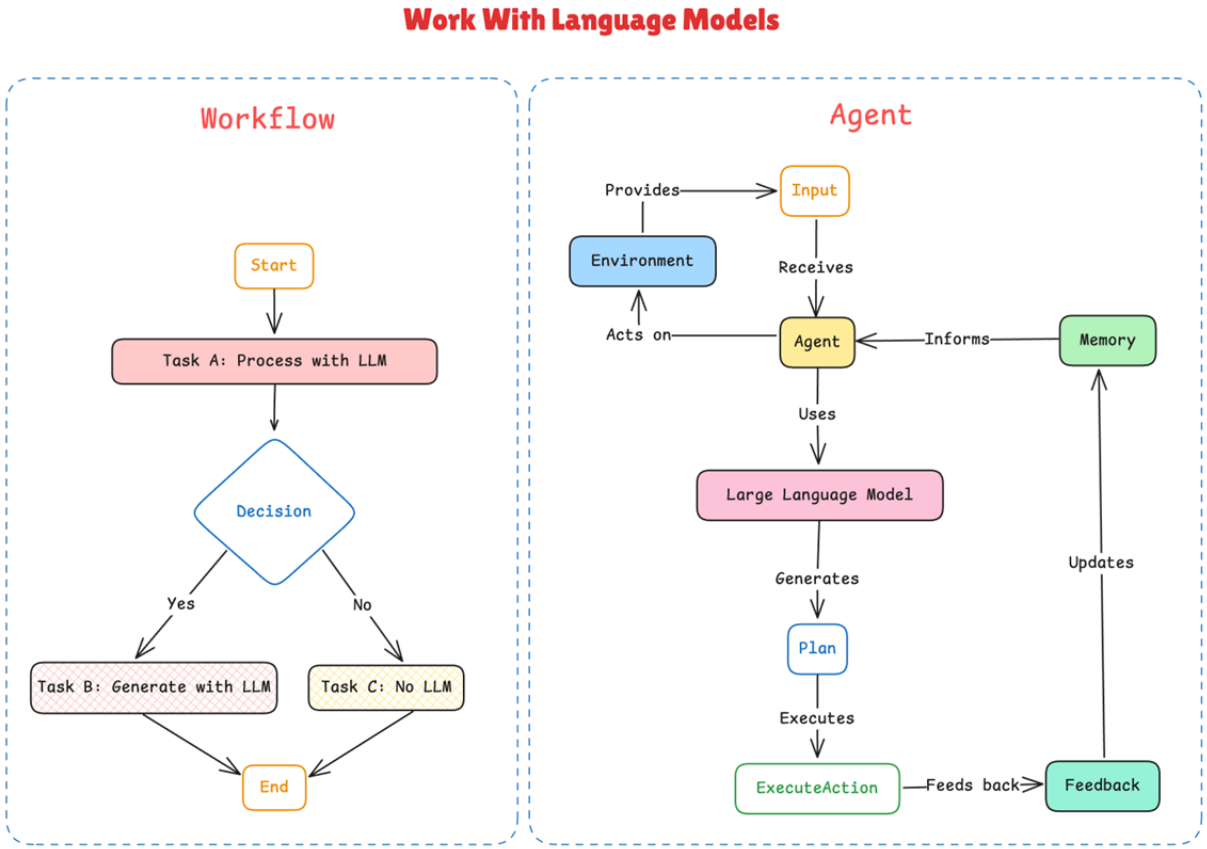


Agents

An LLM that uses your tools

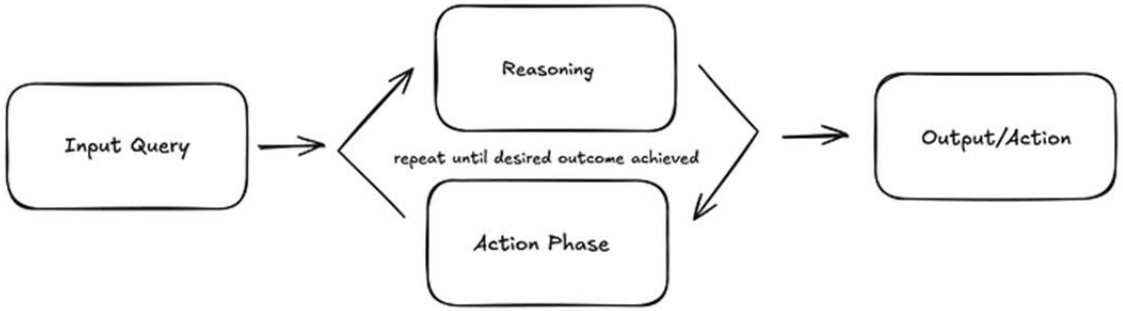
Agent vs model vs workflow



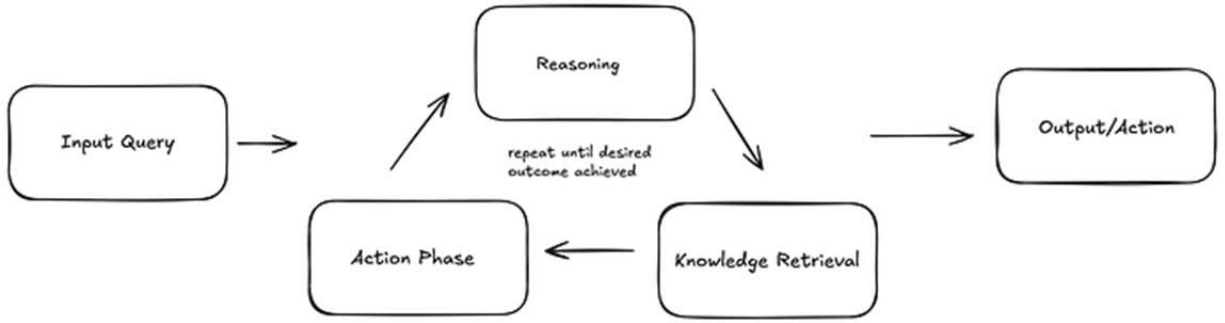


ReAct Agents alternates between reasoning and acting

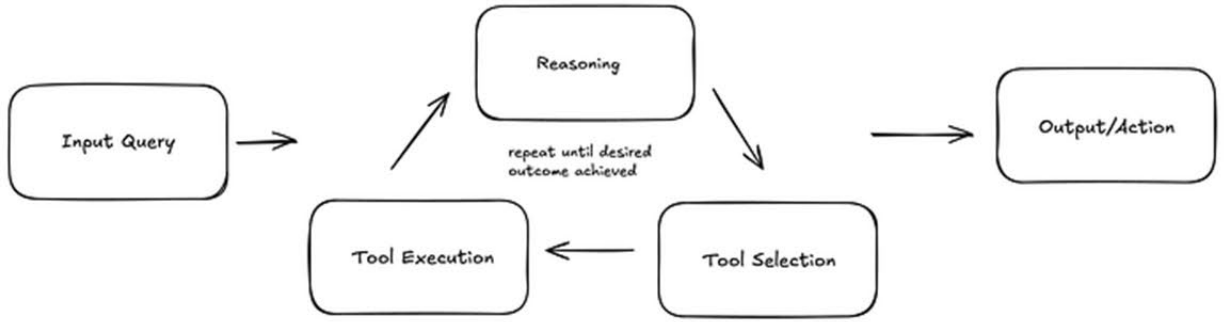
ReAct — Reasoning Meets Action Agent



ReAct + RAG Agent



Tool Enhanced Agent



Our agent's tools are the models you built this week

predict_bulk_modulus

Random forest + uncertainty + in-domain flag

Sessions 2-4

lookup_dft_bulk_modulus

Exact value if the composition is known

Session 1

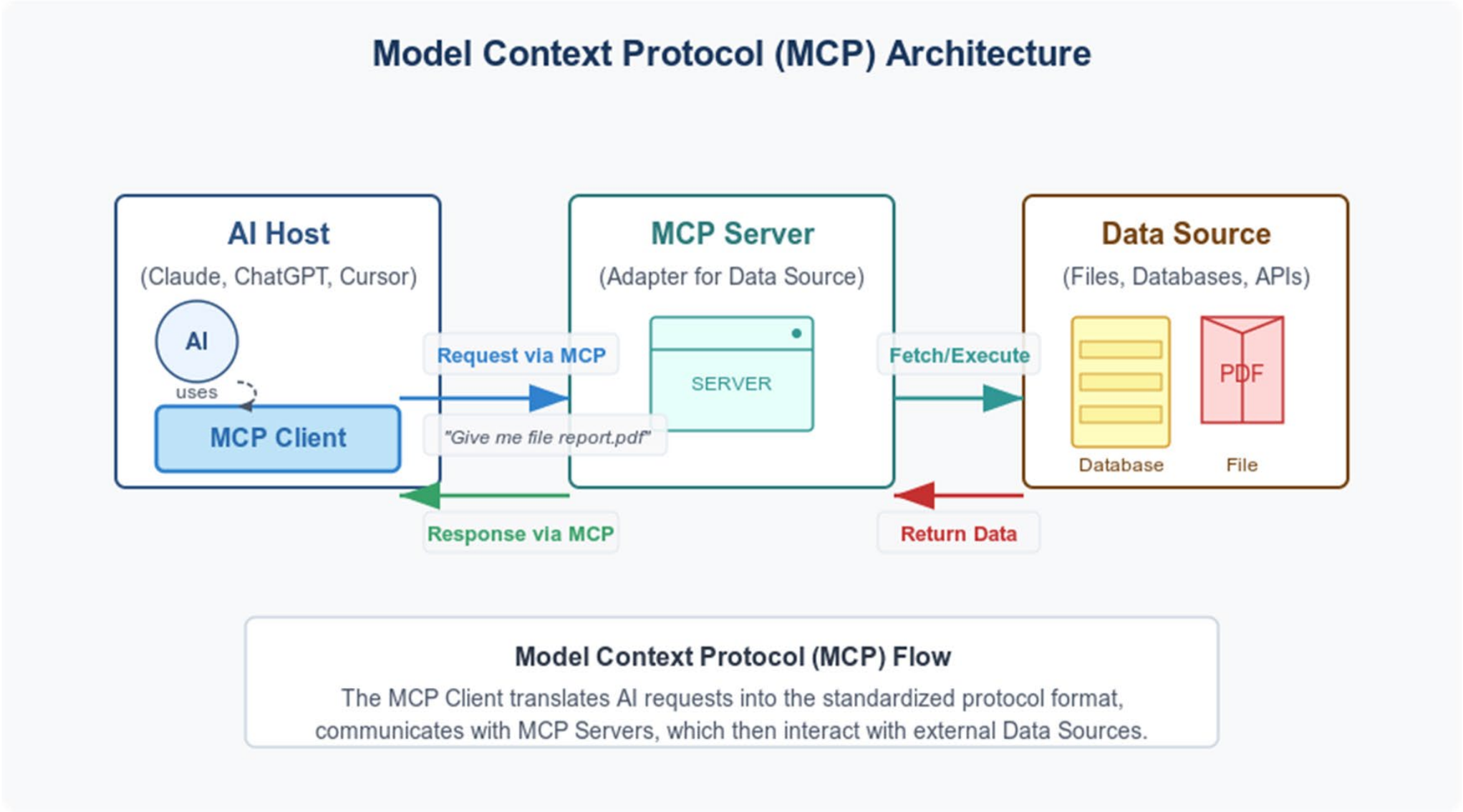
search_literature

Retrieval over the paper corpus

Part B

The answer is only as good as the tools the agent calls

Model Context Protocol (MCP) Architecture



Use a workflow when you know the steps; an agent when you don't

Workflow

Steps fixed in code

Cheaper, faster, easier to validate

Featurize → predict → rank

Agent

The model picks the steps

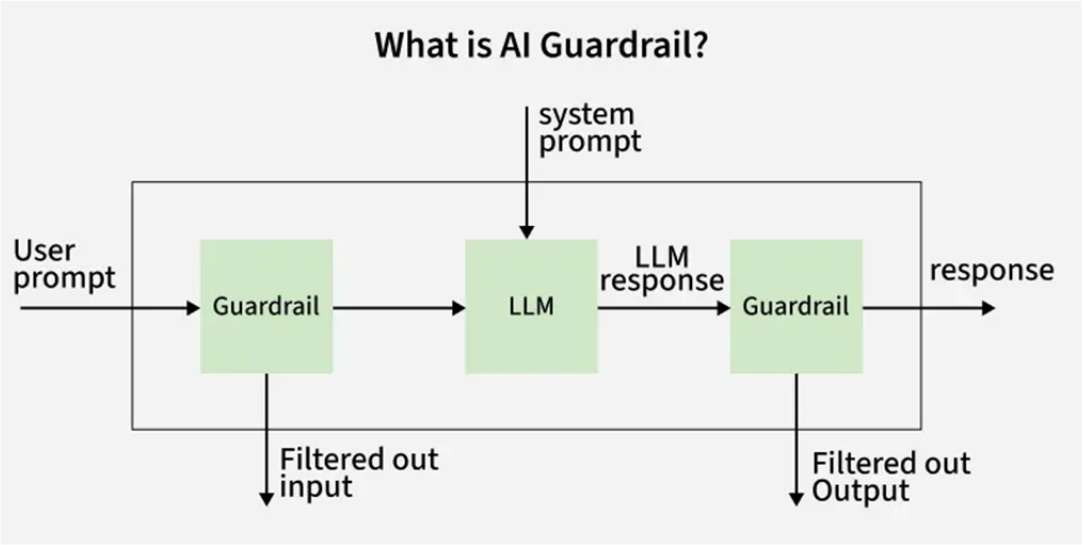
Flexible on open questions

Needs guardrails and logs

Start with a workflow; promote it to an agent only when the path varies

Guardrails for MAS because more agents means more failure modes

- Max iterations to stop runaway loops
- Schema validation for tool calls (e.g. pydantic)
- Range checks for physical plausibility
- Topic filters for off-task behavior
- Confidence gates for uncertain answers
- Human-in-the-loop for high-stakes actions



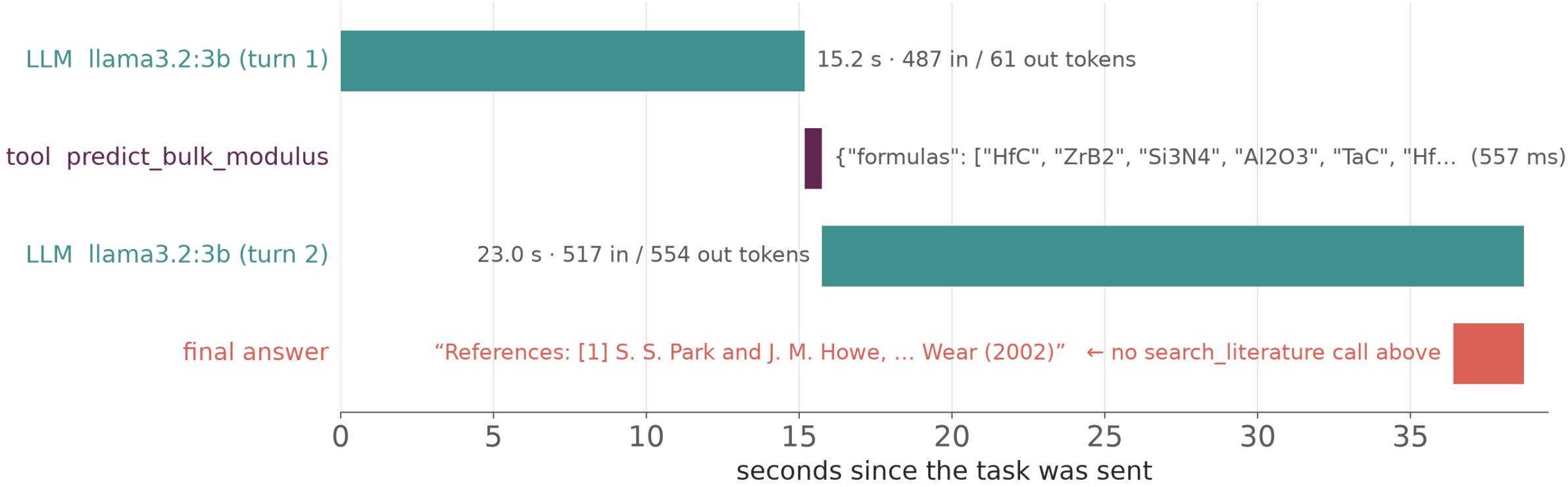
You rarely write the agent loop yourself: there's a mature stack

Models	Cloud: Claude, GPT, Gemini Local: Llama, Qwen, Gemma, Mistral via Ollama
Frameworks	LangGraph / LangChain · LlamaIndex · PydanticAI · smolagents · CrewAI · Claude Agent SDK · OpenAI Agents SDK
Connectors	MCP: tools and data, write once, use from any app · A2A: agents talking to agents
Tracing + evals	LangSmith · Langfuse (open source, self-hostable) · Arize Phoenix · W&B Weave · MLflow

Our notebook's 30-line loop is what all of these do inside. Learn it once, then use a framework

The trace shows the agent never searched, yet it cited three papers

2 model calls, 1 tool call, 39 s. Its three references are invented, and HfC is misranked



Notebook Part C agent and tools, run with llama3.2:3b in Ollama on a 4 GB laptop GPU; trace drawn in the style of LangSmith/Langfuse. Made for this course.

Our agent in LangChain, on a local model, traced in LangSmith

```
# pip install langchain langchain-ollama
# tracing: free LangSmith account at smith.langchain.com
import os
os.environ["LANGSMITH_TRACING"] = "true"
os.environ["LANGSMITH_API_KEY"] = "..."

from langchain.agents import create_agent
from langchain_ollama import ChatOllama

agent = create_agent(
    model=ChatOllama(model="qwen3:4b", temperature=0),
    tools=[predict_bulk_modulus,     # the notebook's
           lookup_dft_bulk_modulus,  # own Python
           search_literature],       # functions
    system_prompt=AGENT_SYSTEM)
result = agent.invoke(
    {"messages": [{"role": "user", "content": TASK}]})
```

Same tools, no rewrite

Plain Python functions with docstrings become tools

Swap one line for the cloud

ChatAnthropic(model="claude-opus-5-5")

Can't send traces out?

Langfuse or Phoenix run on your own server

We ran this code (minus the LangSmith key) on the notebook's task: it ran. The 4B model called the DFT lookup 12 times, never searched, and added hardness values no tool gave it

LLM agents already plan and run chemistry experiments

Coscientist

GPT-4 + web search, documentation, code execution and lab automation

Planned and optimized palladium cross-coupling reactions on real hardware

Boiko et al., Nature 624, 570 (2023)

ChemCrow

GPT-4 + 18 expert chemistry tools

Planned and executed syntheses of an insect repellent and three organocatalysts; guided discovery of a new chromophore

Bran et al., Nat. Mach. Intell. 6, 525 (2024)

KnowMat (ours)

LangGraph multi-agent pipeline (parse, extract, evaluate, flag), traced in LangSmith

Turns papers into a validated materials database

Sayeed et al., IMMI (2026)

In every case the LLM plans; validated tools and instruments produce the numbers

https://doi.org/10.1038/s41586-023-06792-0 · https://doi.org/10.1038/s42256-024-00832-8 · https://doi.org/10.1007/s40192-026-00455-4

Foundation models

Trained once on everything, useful for many tasks

A foundation model is pretrained broadly, then reused

Universal potentials

CHGNet, MACE-MP, MatterSim

Energies and forces in milliseconds

Generative models

MatterGen, CrystaLLM, CrysText

Propose new crystals

Pretrained encoders

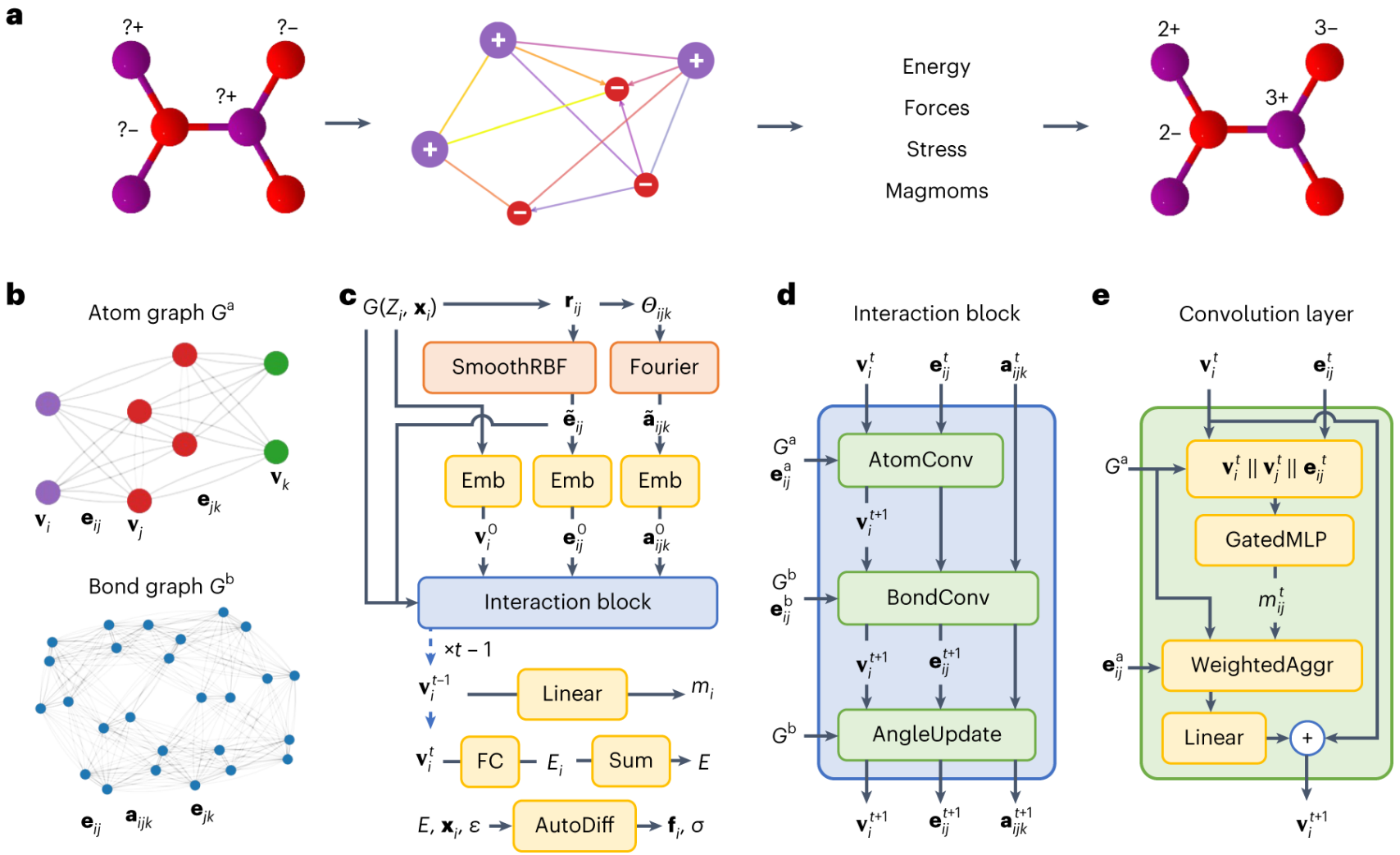
MicroNet for micrographs

LLM embeddings for text

Pretrain broadly → test on your problem → fine-tune if needed

CHGNet predicts energy, forces, stress and magnetic moments

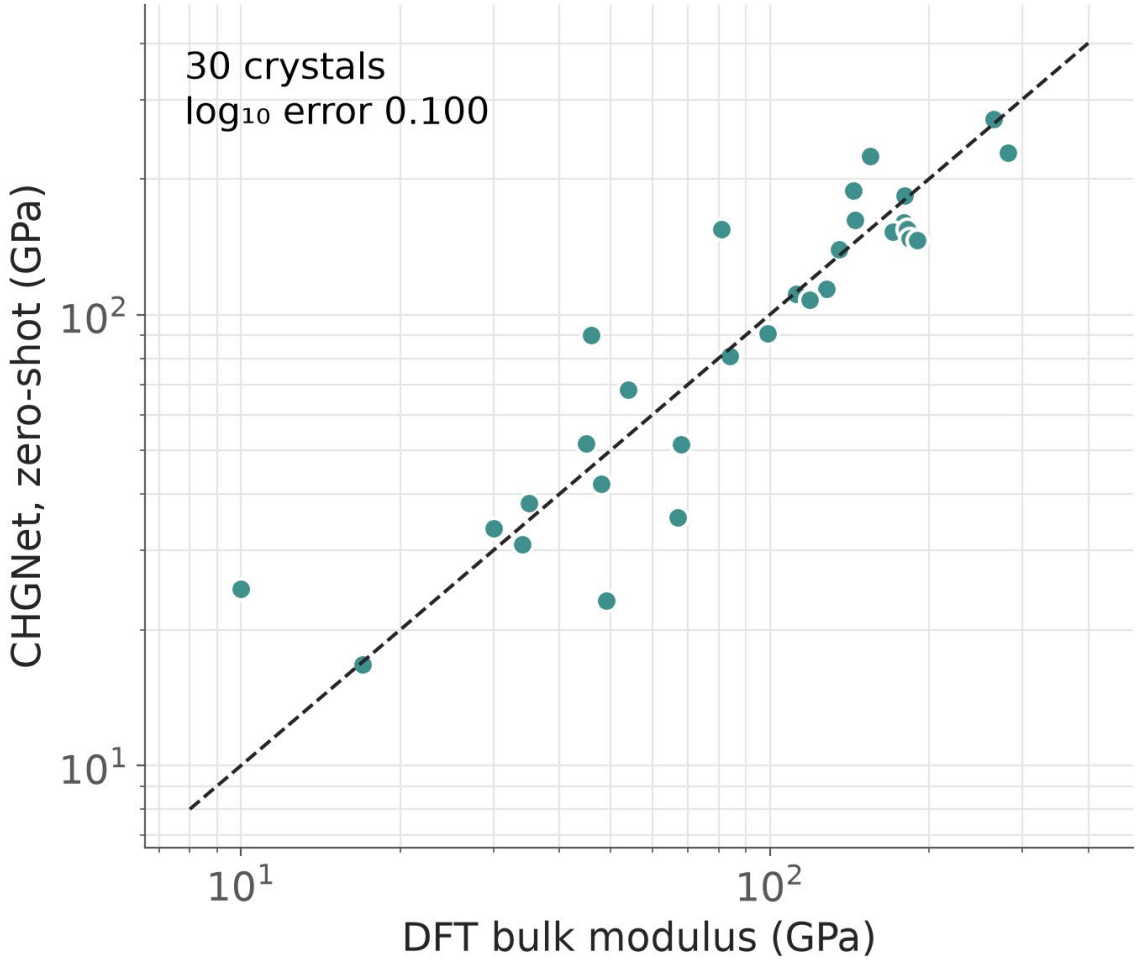
400,438 parameters; test MAE of 30 meV/atom energy and 77 meV/A force on MPtrj



Deng et al., Nat. Mach. Intell. 5, 1031 (2023) https://doi.org/10.1038/s42256-023-00716-3

Zero-shot, CHGNet predicted bulk moduli within about 25%

30 Materials Project crystals: squeeze ±2%, fit an equation of state, compare with DFT



Structures and DFT moduli: Materials Project via Matbench (Dunn et al. 2020). CHGNet: Deng et al., Nat. Mach. Intell. 2023. Analysis made for this course.

A zero-shot model nearly matched ones trained for the task

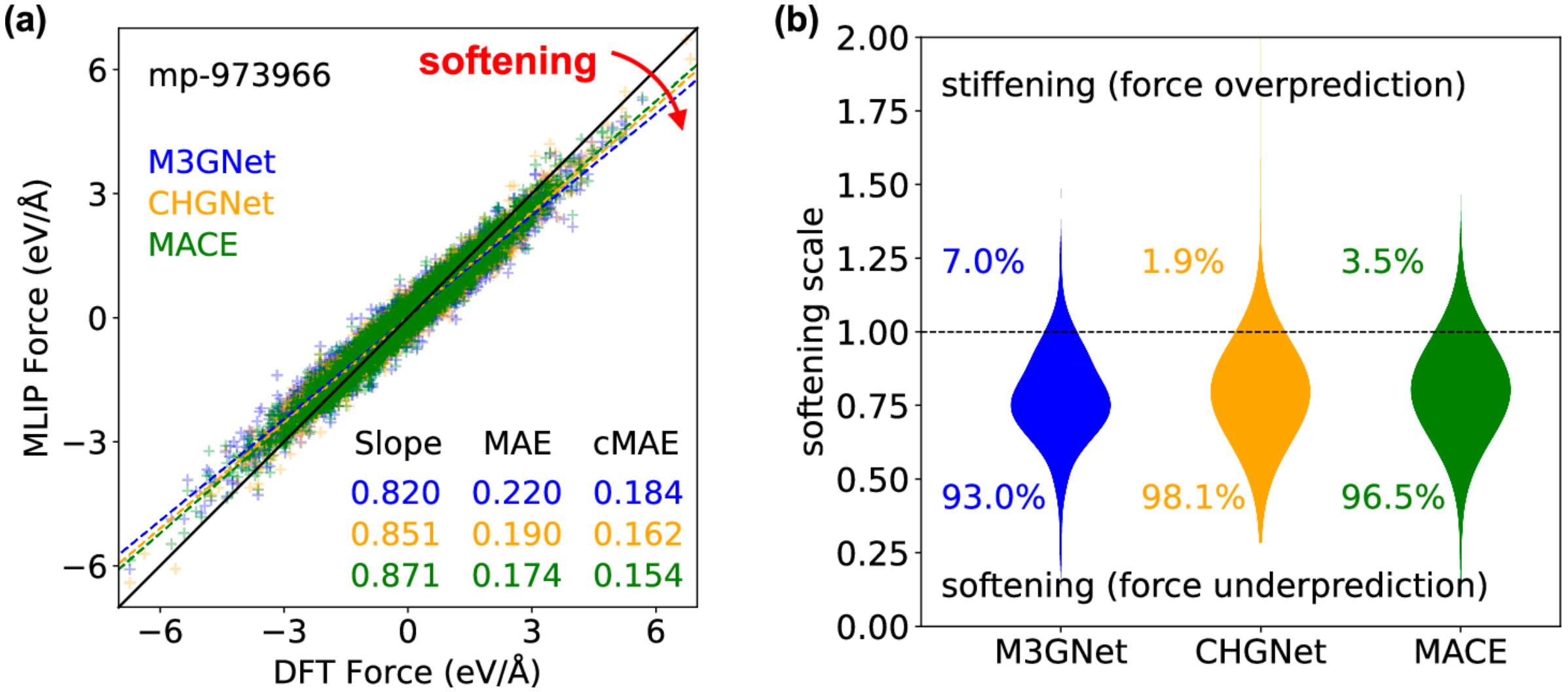
Same property, same data source; the first two were trained on it in Session 2



Course analyses, Sessions 2 and 6.

Universal MLIPs systematically underpredict forces

Over 90% of 1,000 unseen compounds showed softening in all three models (slope below 1)



Deng et al., npj Comput. Mater. 11, 9 (2025) https://doi.org/10.1038/s41524-024-01500-6

We have so many generative models to choose from!

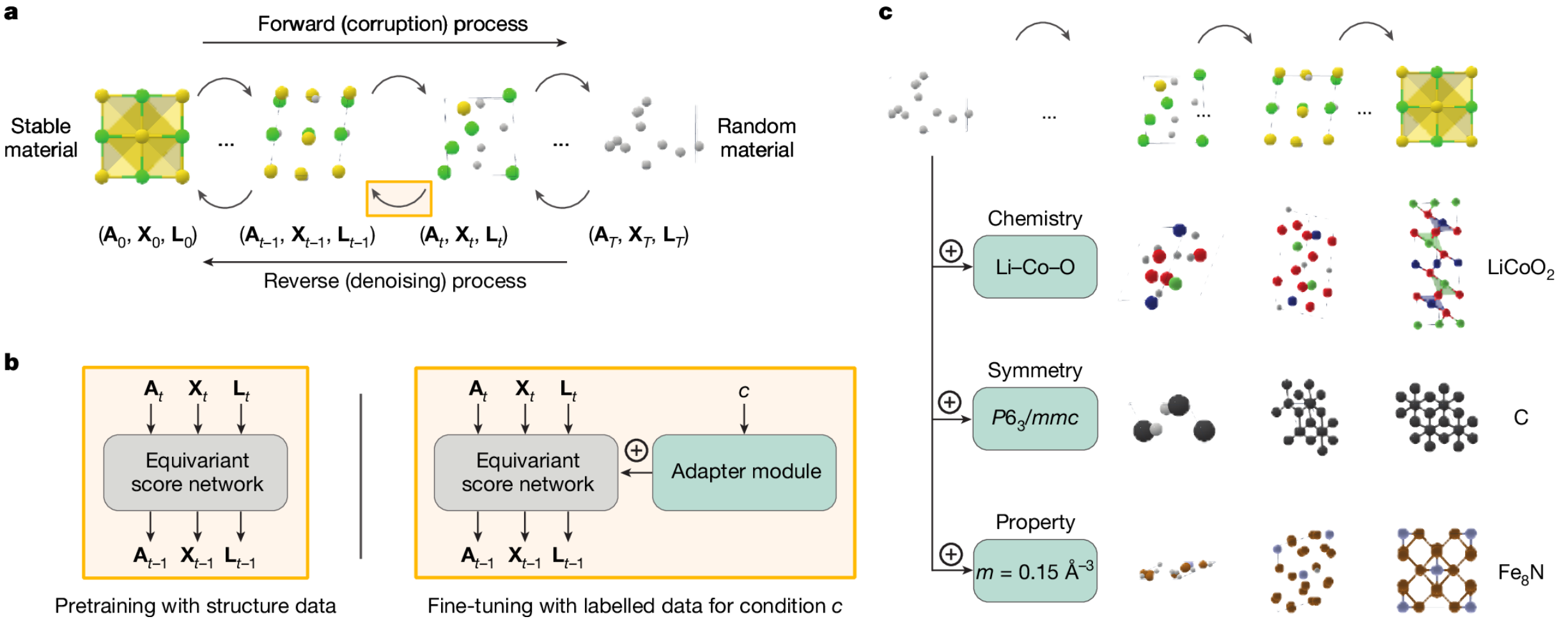
Table 1. Selection of generative models developed for crystalline inorganic materials

Model	Structure representation	Generation architecture	License	Reference
CrystalGAN[a]	coordinates	generative adversarial network (GAN)	GPL	Nouira et al.[17]
Hoffmann et al.[b]	voxels	variational autoencoder (VAE)	MIT	Hoffmann et al.[18]
iMatGen[c]	voxels	VAE	–	Noh et al.[19]
CCCGAN[d]	coordinates	GAN	–	Kim et al.[20]
ICSG3D[e]	voxels	VAE	MIT	Court et al.[21]
CCDCGAN[f]	voxels	GAN	–	Long et al.[22]
CubicGAN[g]	coordinates	GAN	MIT	Zhao et al.[23]
FTCP[h]	coordinates and structure factors	VAE	Apache	Ren et al.[24]
CDVAE[i]	coordinates	VAE + diffusion	MIT	Xie et al.[25]
CrystaLLM[j]	crystallographic information file	autoregressive transformers (ATs)	MIT	Antuneset al.[26]
xyzTransformer[k]	crystallographic information file	ATs	–	Flam-Shepherdet al.[27]
✅ CrysTens[l]	coordinates	GAN, diffusion	MIT	Alverson et al.[28]
DiffCSP[m]	coordinates	diffusion	MIT	Jiao et al.[29]
MatterGen[n]	coordinates	diffusion	–	Zeni et al.[30]
WyCryst[o]	Wyckoff site matrix	VAE	–	Zhu et al.[31]
UniMat[p]	coordinate embedded periodic table	diffusion	–	Yang et al.[32]
Crystal-llm[q]	coordinates	ATs	CC-BY-NC	Gruver et al.[33]

Park, Hyunsoo, Zhenzhu Li, and Aron Walsh. "Has generative artificial intelligence solved inverse materials design?." *Matter*7.7 (2024): 2355-2367.

Diffusion models can generate crystals with target properties

Over twice as likely to give stable, new structures as prior models, and over 10x closer to DFT minima



Zeni et al., Nature 639, 624 (2025) https://doi.org/10.1038/s41586-025-08628-5

Our LLM approach to crystal generation is published in IMMI

SPRINGER NATURE Link

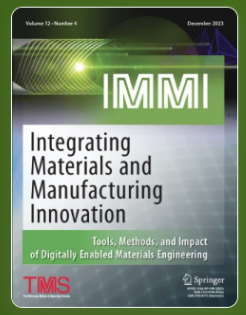
Find a journal | Publish with us | Track your research | Search | Saved research

Home > Integrating Materials and Manufacturing Innovation > Article

CrysText: A Generative AI Approach for Text-Conditioned Crystal Structure Generation Using LLM

Technical Article | Published: 21 March 2026

(2026) Cite this article



Integrating Materials and Manufacturing Innovation

Aims and scope →

Submit manuscript →

Save article

Trupti Mohanty, Maitrey Mehta, Hasan M. Sayeed, Bat-El Oded, Itay Pitussi, Arie Borenstein, Vivek Srikumar & Taylor D. Sparks

96 Accesses Explore all metrics →

Access this article

Log in via an institution →

SCIENCE TO SHAPE THE FUTURE OF THE ARMY
ARMY RESEARCH OFFICE
1951

Local or cloud?

Choosing an LLM for your data

Local or cloud: decide on data, quality and volume

	Cloud frontier model	Local open model
Data leaves your network	Yes (check retention terms and policy)	No
Quality on hard extraction	Highest	Lower, improving fast
Cost	Per token, no hardware	GPU, setup, maintenance
Best for	Prototypes, hard documents, agents	Confidential data, huge volume

Prototype in the cloud, then score a local model on the same answer key

Extracting a thousand paragraphs costs dollars, not thousands

Claude Opus 5.5	Claude Haiku 5.5
≈ $15	≈ $0.37
per 1,000 paragraphs	per 1,000 paragraphs

The expensive part is checking the output, not generating it

Ollama runs open models on your own machine in two commands

```python
# Python: notebook Part A, run locally
import json, ollama
r = ollama.chat(
    model="qwen3:4b",
    messages=[{"role": "system", "content": EXTRACTION_SYSTEM},
              {"role": "user", "content": paragraph}],
    format=EXTRACTION_SCHEMA,  # same schema as Claude
    options={"temperature": 0}, think=False)
records = json.loads(r.message.content)["records"]
```

```
# terminal (installer: ollama.com/download)
ollama pull qwen3:4b   # 2.5 GB, once
ollama run qwen3:4b "Hi"
ollama list            # what you have
```

Models we use: Qwen3, Gemma 3, Llama 3.x, Mistral, gpt-oss, DeepSeek-R1 distills
New optional cell in notebook Part F runs this in Colab and scores it

Nothing leaves your machine

Prompts, papers and results stay local

OpenAI-compatible API

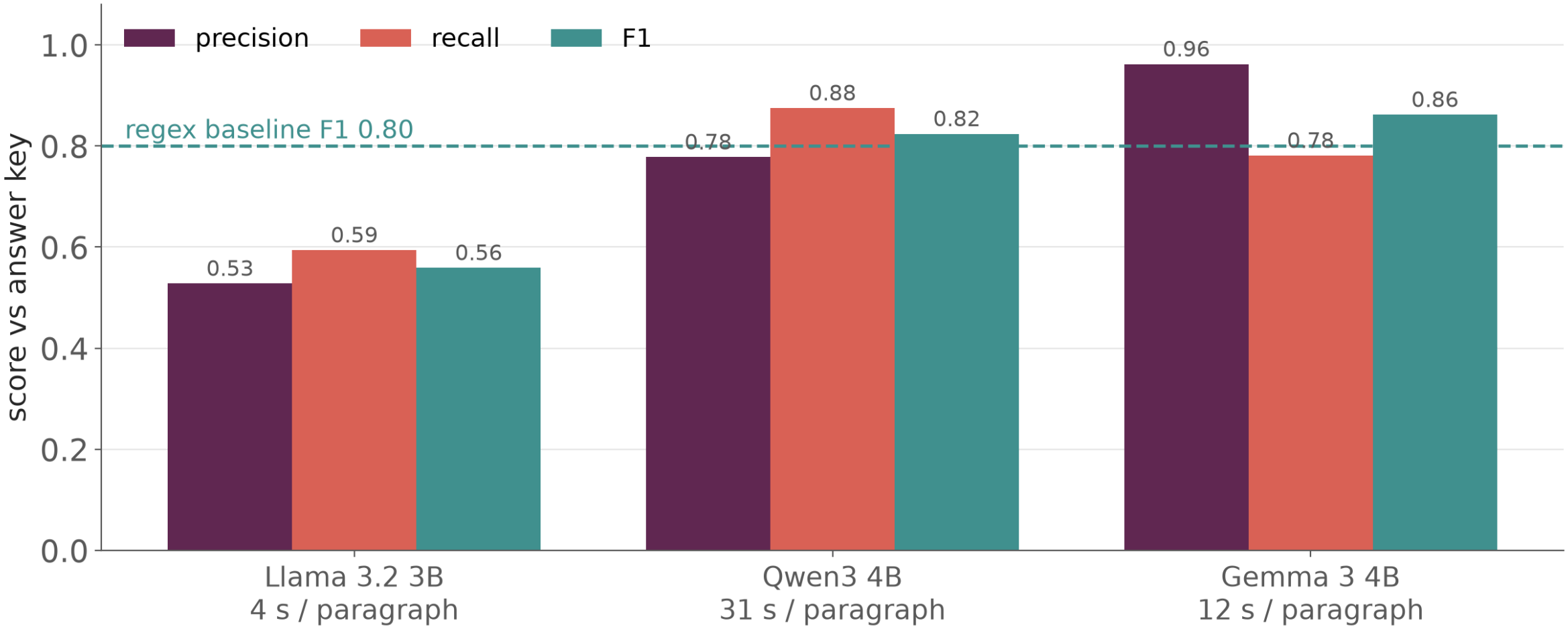
localhost:11434/v1: LangChain, LlamaIndex, Open WebUI

Embeddings too

nomic-embed-text: the embedding slide ran locally

Small local models: F1 0.56-0.86 on our answer key; the regex scored 0.80

Score a local model before trusting it. Unlike regex, the LLMs also say which material each value belongs to



Notebook Part A answer key (32 records). Ollama, 4-bit models, temperature 0, RTX 3050 Ti laptop GPU (4 GB). Made for this course.

Memory decides what you can run: about 0.6 GB per billion parameters

Laptop

4-8 GB GPU or 16 GB RAM

3-8B models

Extraction drafts, embeddings, private chat

Workstation

24 GB GPU or 64 GB Mac

14-32B models

Stronger reasoning, longer documents

Server

One or more 80 GB GPUs

70B+ models

Serve your whole group with vLLM

Ollama

CLI + API, easiest

LM Studio

desktop app, GUI

llama.cpp

the engine inside

vLLM

fast multi-user server

Open WebUI

ChatGPT-style page for your lab

Sizes measured on this laptop: 3B = 2.0 GB, 4B = 2.5-3.4 GB, 7-8B = 4.1-4.7 GB (4-bit)

Knowledge graphs store what LLMs extract, with structure

Extract

LLM / KnowMat pulls facts from papers

Normalize

Ontology: one name per material, unit, method

Store

Graph: material → process → property → paper

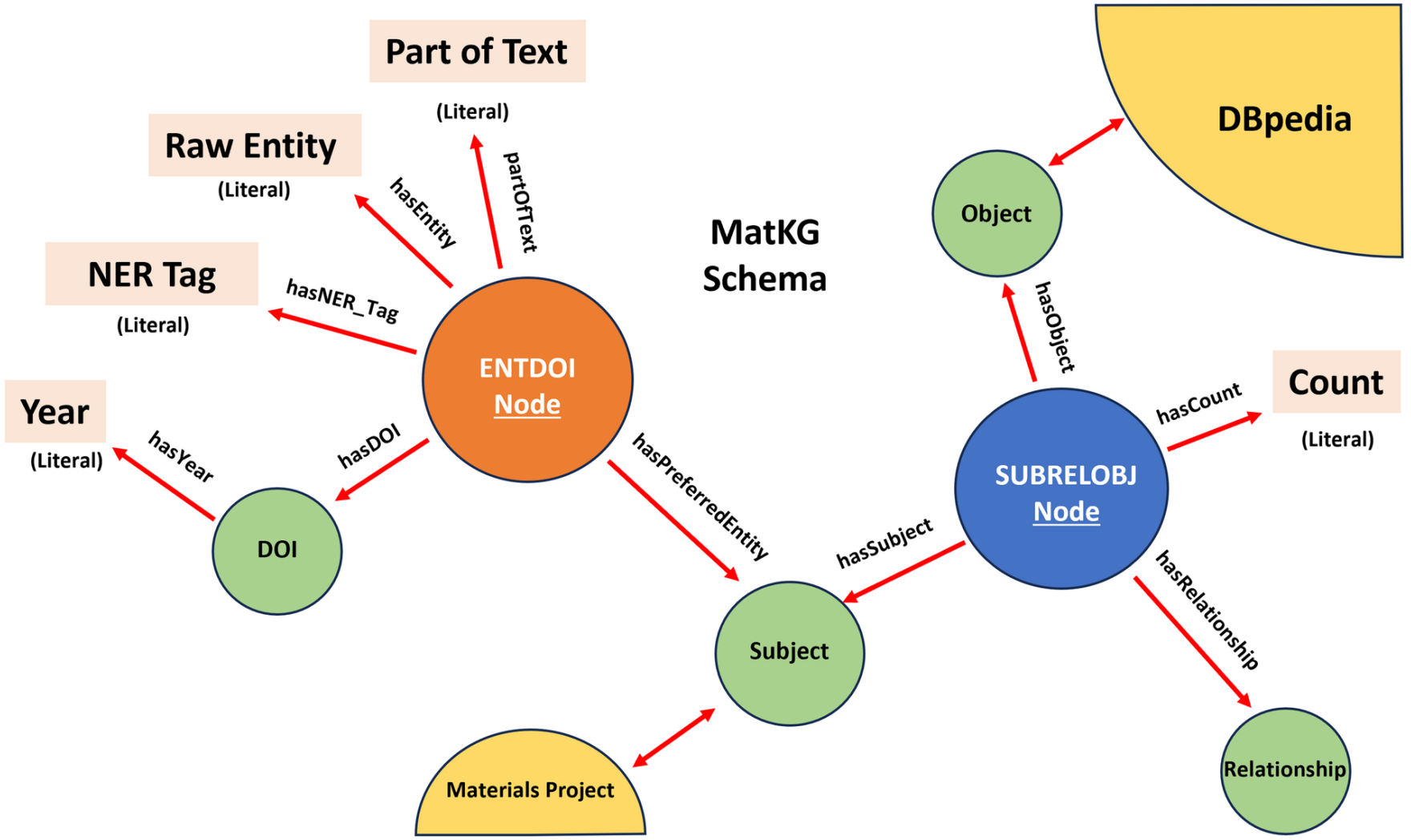
Query

People and agents ask questions with provenance

LLMs read; ontologies keep it consistent; graphs keep it queryable

A knowledge graph turns papers into linked, queryable facts

MatKG: over 70,000 entities and 5.4 million unique triples mined from about 5 million papers



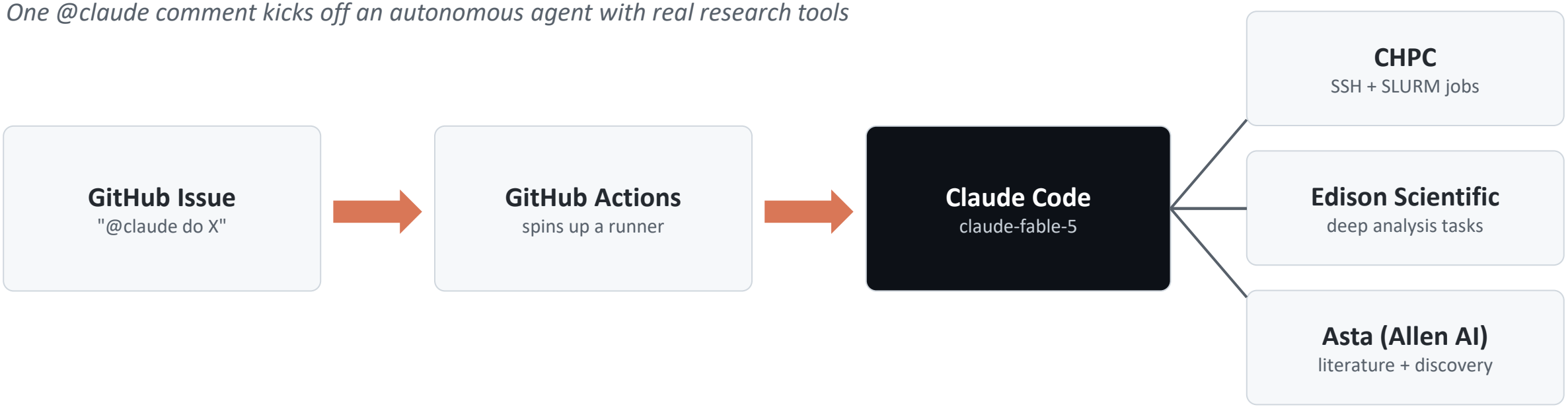
Venugopal & Olivetti, Sci. Data 11, 217 (2024) https://doi.org/10.1038/s41597-024-03039-z

How this course was built

An AI coding agent, checked like a thesis

What We're Building

One @claude comment kicks off an autonomous agent with real research tools



Results come back as commits, plots, and comment replies in the issue thread

The key idea: all credentials live as encrypted secrets in ONE repo. Anyone with write access can put the agent to work. Nobody else touches a token.

The prompts were short; the checking was not

"Prepare the 3rd slide deck in the same format style we've been using and prepare a good notebook that guides them to building, training, evaluating and using models."

an actual prompt for this course

Run everything

Every notebook executed end to end before publishing

Numbers from code

Slides regenerated from the same data and seeds

Surprises stay in

If a result disagreed, the text changed, not the data

A human signs off

Licenses, numbers and decks reviewed

4 Add CLAUDE.md: The Agent's Onboarding Doc

Read automatically at the start of every run. This is where the magic lives

Tool playbooks

How to call Edison (foreground polling only!), when Asta is worth the credits, CHPC login node + SLURM etiquette

Safety rails

Never echo/grep secrets. Never run heavy jobs on login nodes. rsync carefully

House style

How to embed plots in comment replies, link files by commit hash, format issue references

Failure protocol

What to report when a token expires or a run blocks on Duo 2FA

Rule of thumb: every time the agent does something dumb, don't just fix it, add a line to CLAUDE.md so it never happens again. It compounds.

GitHub Actions vs. CLI: Which Way to Run Your Agents?

Honest scorecard, I use both

GitHub Actions agent

✓ **Credentials once, whole team works. Lab members trigger agents with a comment: no setup, no shared tokens, no Anthropic accounts**

✓ Everything is auditable: every run logged, every change a commit

✓ Async and parallel: file 5 issues, walk away, review results later

✓ Runs from your phone: a GitHub comment is the whole interface

✗ Slow feedback loop: no mid-task steering, wait for the run

✗ Runner limits: 2-core VM, wall-clock ceilings, no GPU

✗ Debugging means reruns: tweak, push, wait, read logs, repeat

Claude Code CLI (local)

✓ Tight loop: watch the agent think, steer and interrupt in real time

✓ Your machine, your hardware: local files, GPUs, no time limits

✓ Best for exploration: figuring out what the task even is

✓ Interactive auth just works: Duo pushes, browser logins

✗ **Every user needs full setup: CLI, plugins, and their own copy of every credential**

✗ Ties up your machine and attention while it runs

✗ No shared audit trail unless you're disciplined about commits

Choosing methods with confidence

For newcomers, and for taking this to work

A good question with honest validation beats a novel method

Start with the question

What decision will this change?

Start simple

Baseline, then random forest, then fancier

Validate honestly

That's what reviewers and managers trust

Explain what you learned

Physics, not just R^2

Methods change every year; these habits don't

Your first AI project at work: six steps

1. Pick a decision
Repeated, valuable, currently slow

2. Inventory the data
What exists, in what shape, who owns it

3. Beat a baseline
Mean, rule of thumb, current practice

4. Pilot small
One product line, a few weeks, a clear metric

5. Measure the payoff
Experiments saved, time to target

6. Then build infrastructure
Data capture that makes the next model easy

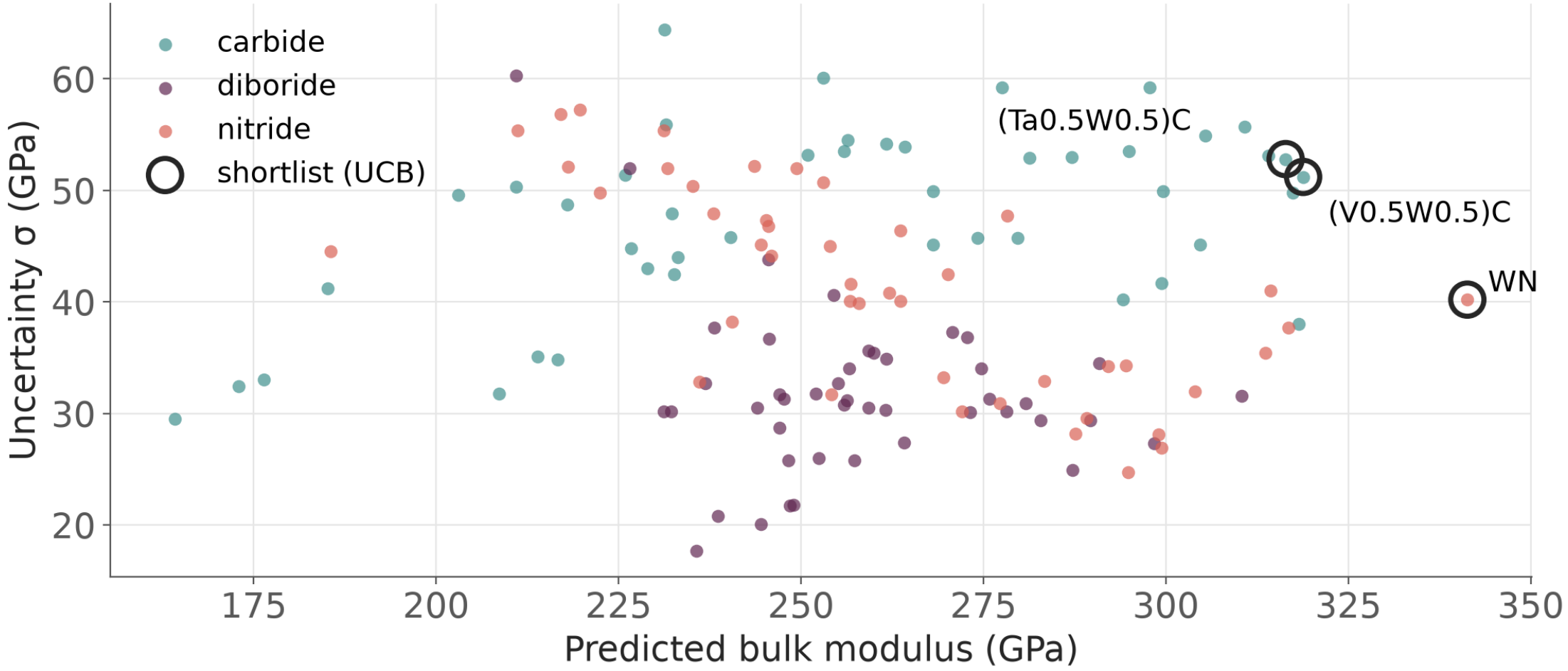
Most failures are framing and data, not algorithms (Session 1)

Capstone: data → model → decision

Closing the loop

Capstone: three stiff carbides and nitrides worth making next

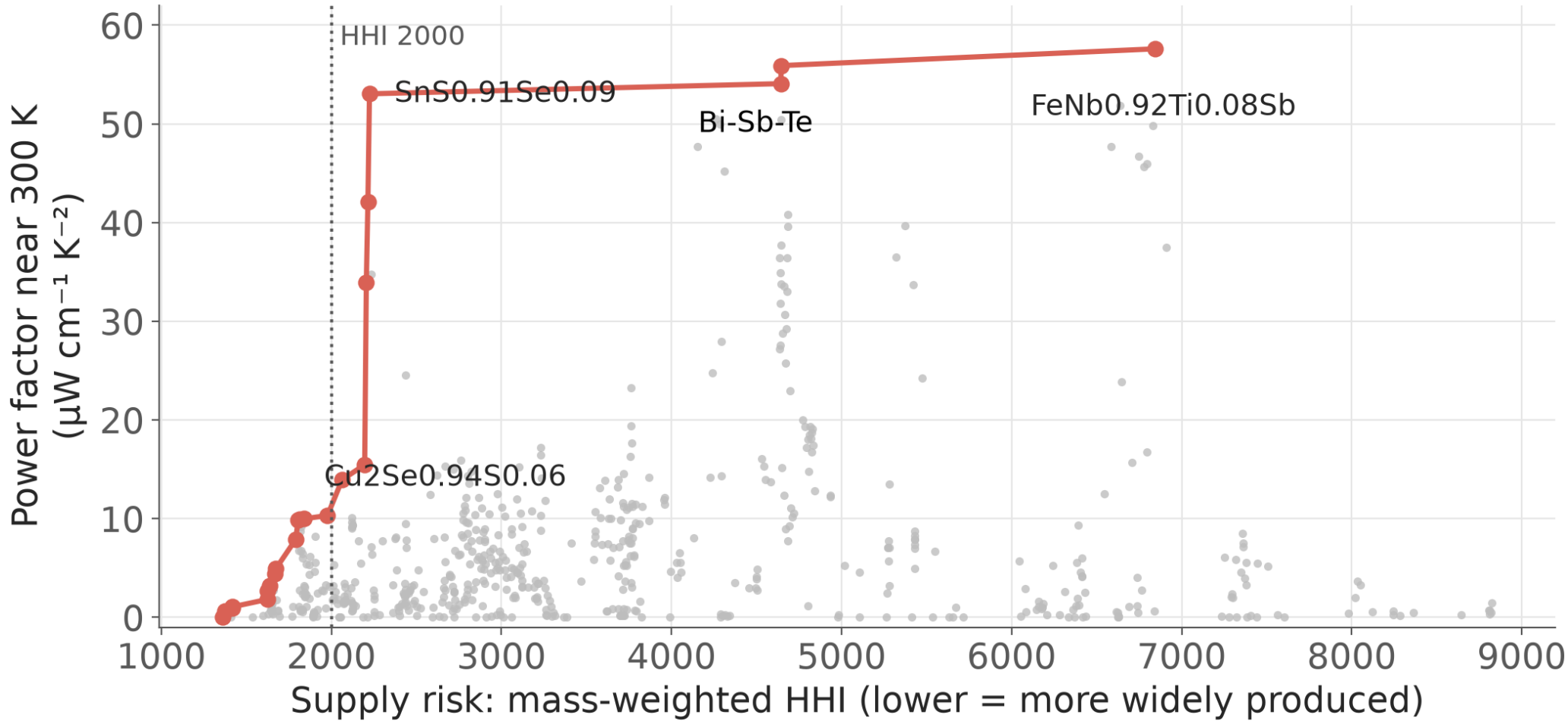
135 candidates; 24 already known excluded; ranked by predicted modulus + uncertainty (UCB)



Course data and models (Sessions 2-5). Analysis made for this course.

Thermoelectric track: the Pareto front becomes a recommendation memo

Notebook D2: the LLM drafts; you check every number against the table



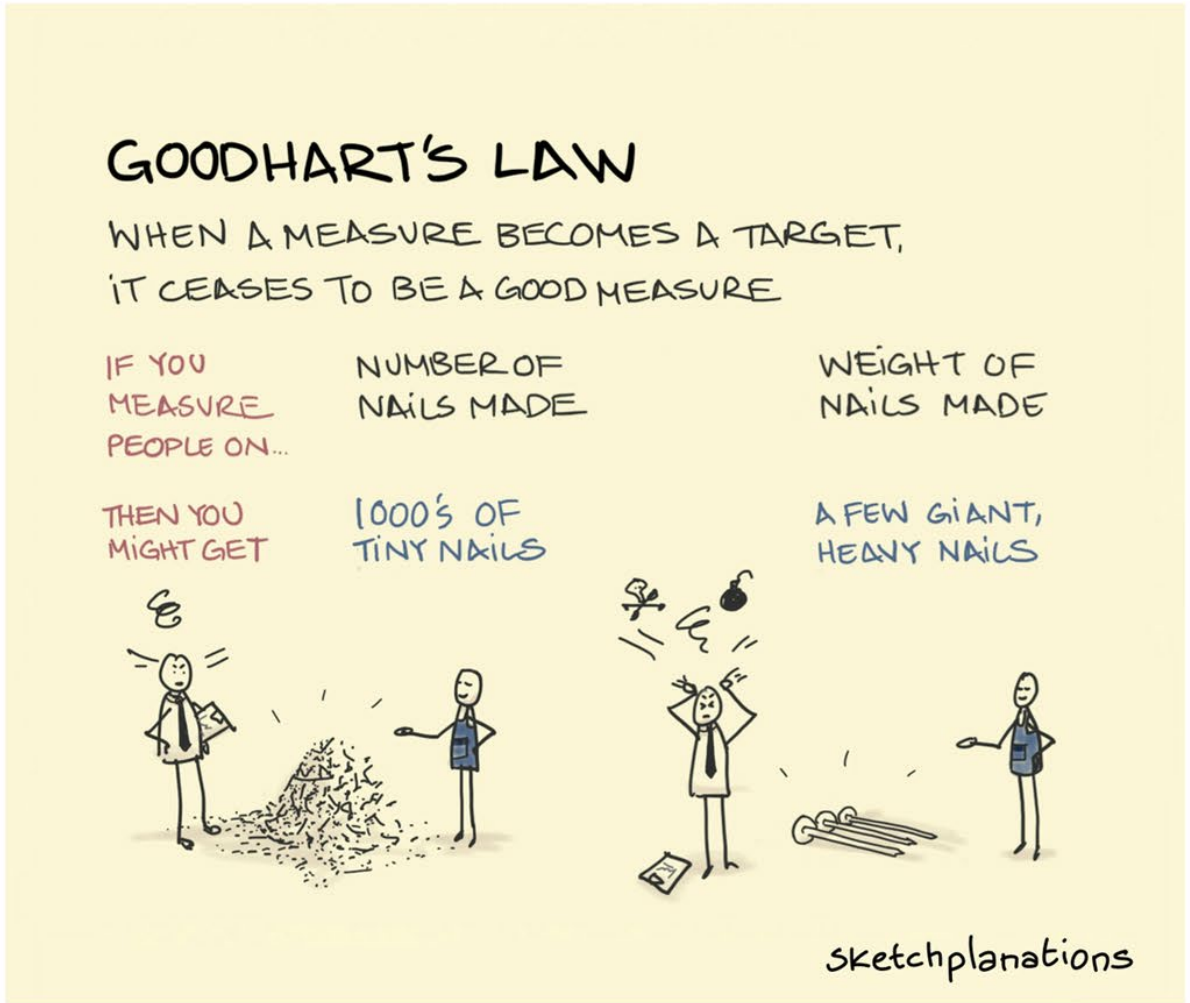
Data: sysTEm (Tang et al., IMMI 2026); HHI: Gaultois et al. 2013. Analysis made for this course.

Pick your track: the loop is the same

Track	Data	Model	Decision
Stiff ceramics	DFT bulk moduli	CBFV + forest + uncertainty	UCB shortlist
Thermoelectrics	Experimental sysTEm	Measured PF + supply risk	Pareto memo
Formulations	Your recipes	Gaussian process	Batch BO suggestions
Micrographs	Your images	Pixel classifier → grain size	Which samples next
Literature → data	Your PDFs	LLM extraction, scored	Which values to trust

Every track ends with a decision a human signs

Goodhart's Law



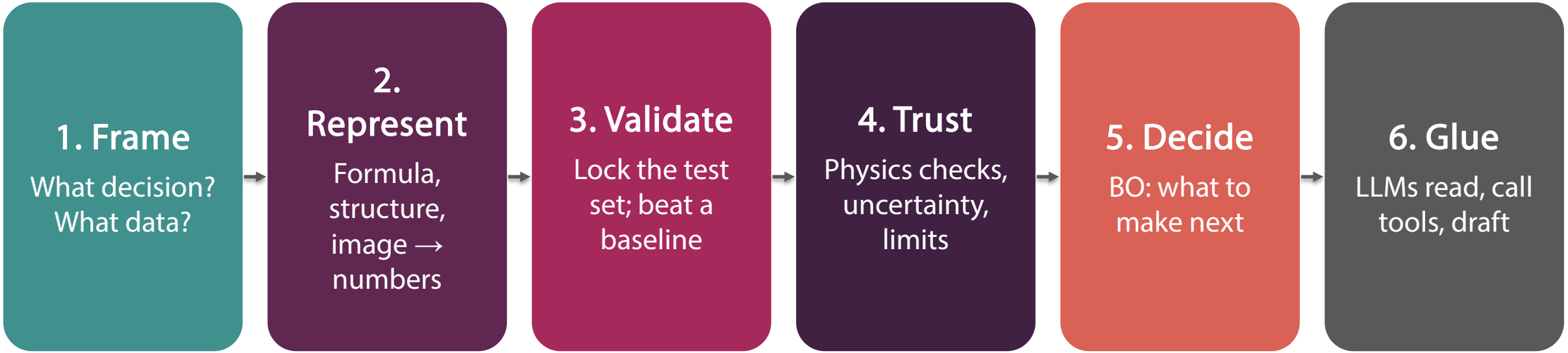


Copyright © 1999 United Feature Syndicate, Inc.
Redistribution in whole or in part prohibited

Closing: don’t jump to conclusion from now on!

“AI slop” is often less about AI itself or the specific model, and more about how people use these tools without thinking about design principles.

The whole course in one loop



Data → model → decision, with a human checking every step

Four things to remember from today

1. LLMs are glue

Numbers come from validated models, not the LLM

2. Score your extraction

An answer key, evidence quotes, a random spot-check

3. Foundation models are free baselines

Test them on your problem before trusting

4. A human signs off

On memos, on data, on decisions

Thank you, and keep in touch: github.com/sp8rks/ACerS_AI_ML_Workshop

Capstone hands-on

45 minutes · Google Colab · API key optional

Open the Session 6 notebook in Google Colab

1. **Go to the course repo**
   github.com/sp8rks/ACerS_AI_ML_Workshop

2. **Click 'Open in Colab'**
   Session 6: 06_llms_and_agents

3. **Optional: add ANTHROPIC_API_KEY**
   Colab 🔑 Secrets; otherwise offline mode

4. **Runtime → Run all**
   Then pick your capstone track

Exercises: pick the one closest to your work

LLMs	Models	Capstone
Ask the agent something its tools can't answer	Foundation model on oxides only: same bias?	Change κ: which shortlist would you defend?
Extract from a paragraph of your own	Score a small local model on the key	Write the memo for your own problem

Then sketch the data → model → decision loop for the problem you framed in Session 1